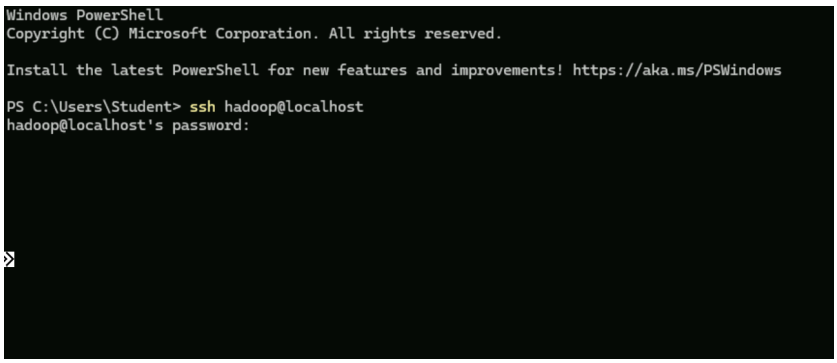
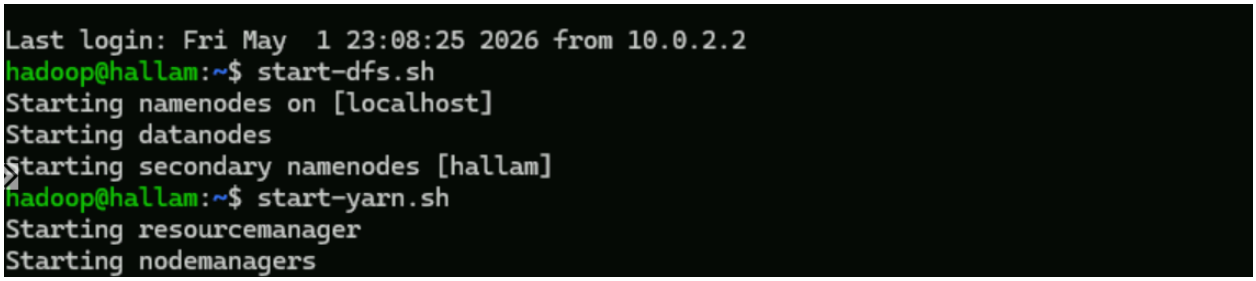
HADOOP $ HIVEQL COMMAND DOCUMENTATION

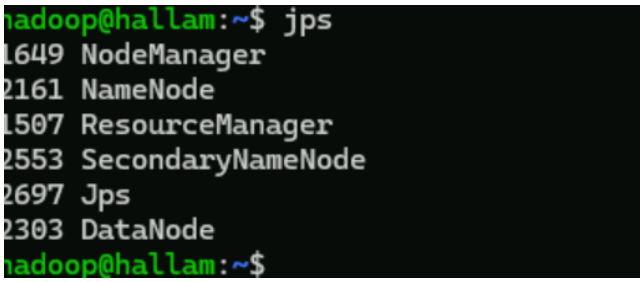
New Command Line Tab:
Connect to Hadoop node:
ssh hadoop@localhost **(Ubuntu Linux Environment)**
Enter Password:
hadoop



Start HDFS and YARN services:
start-dfs.sh **(Start Hadoop)**
start-yarn.sh **(Start YARN ResourceManager and NodeManager)**



Check if Hadoop & Yarn Resources are running:
Jps (most display the namemode, datanode, secondary namenode, resource manager, node managers)



Start MySQL service for Hive Metastore (if not already running):
sudo systemctl start mysql

```
hadoop@hallam:~$ sudo systemctl start mysql
[sudo] password for hadoop:
hadoop@hallam:~$
```

sudo systemctl status mysql **(Check MySQL status if running):**

```
hadoop@hallam:~$ sudo systemctl status mysql
[sudo] password for hadoop:
● mysql.service - MySQL Community Server
     Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
     Active: active (running) since Sat 2026-05-02 09:50:20 UTC; 5h 22min ago
   Main PID: 946 (mysqld)
     Status: "Server is operational"
      Tasks: 79 (limit: 33321)
     Memory: 492.6M
        CPU: 5min 53.376s
     CGroup: /system.slice/mysql.service
             └─946 /usr/sbin/mysqld

May 02 09:50:15 hallam systemd[1]: Starting MySQL Community Server...
May 02 09:50:20 hallam systemd[1]: Started MySQL Community Server.
```

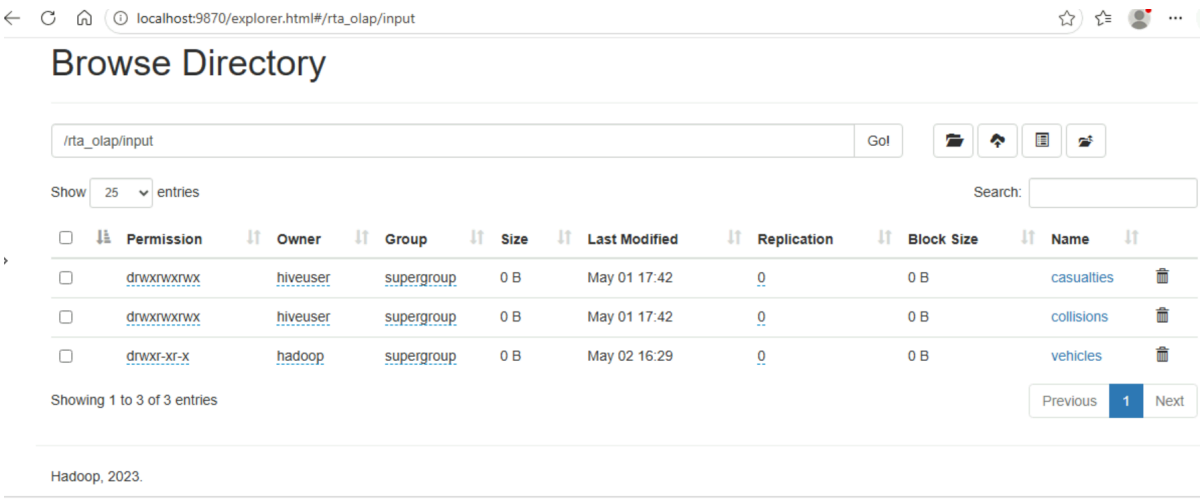
Create an olap folder and path in HDFS:
hdfs dfs -mkdir -p /rta_olap **(Create main project folder)**
hdfs dfs -mkdir -p /rta_olap/input **(Create input folder)**
hdfs dfs -mkdir -p /rta_olap/input/collisions **(Create dataset subfolders:collision)**
hdfs dfs -mkdir -p /rta_olap/input/vehicles **(Create dataset subfolders:collision)**
hdfs dfs -mkdir -p /rta_olap/input/casualties **(Create dataset subfolders:collision)**

Verify on the GUI:

/ Go!

Show 25 entries Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	hadoop	supergroup	0 B	May 01 06:34	0	0 B	project
drwxrwxrwx	hadoop	supergroup	0 B	May 02 16:01	0	0 B	rta_olap
drwxrwxrwx	hadoop	supergroup	0 B	May 01 17:39	0	0 B	tmp



Open a PowerShell Tab:
Copy rta parquet files from windows to Ubuntu:
scp -P 22 "C:\Users\Student\Desktop\rta_olap\Collisions.parquet"
hadoop@localhost:/home/hadoop/rta_olap/input/

scp -P 22 "C:\Users\Student\Desktop\rta_olap\Vehicles.parquet"
hadoop@localhost:/home/hadoop/rta_olap/input/

scp -P 22 "C:\Users\Student\Desktop\rta_olap\Casualties.parquet"
hadoop@localhost:/home/hadoop/rta_olap/input/

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Student> scp -P 22 "C:\Users\Student\Desktop\rta_olap\Collisions.parquet" hadoop@localhost:/home/hadoop/rta_olap/input/
hadoop@localhost's password:
Collisions.parquet                                                                            100% 5097KB  16.7MB/s   00:00
PS C:\Users\Student>
PS C:\Users\Student> scp -P 22 "C:\Users\Student\Desktop\rta_olap\Vehicles.parquet" hadoop@localhost:/home/hadoop/rta_olap/input/
hadoop@localhost's password:
Permission denied, please try again.
hadoop@localhost's password:
Vehicles.parquet                                                                              100% 8349KB  22.1MB/s   00:00
PS C:\Users\Student>
PS C:\Users\Student> scp -P 22 "C:\Users\Student\Desktop\rta_olap\Casualties.parquet" hadoop@localhost:/home/hadoop/rta_olap/input/
hadoop@localhost's password:
Casualties.parquet                                                                            100% 6108KB  16.9MB/s   00:00
PS C:\Users\Student>
```

Load traffic datasets parque files from Ubuntu OS to hdfs warehouse:
hdfs dfs -put -f /home/hadoop/rta_olap/input/Collisions.parquet /rta_olap/input/collisions/

hdfs dfs -put -f /home/hadoop/rta_olap/input/Vehicles.parquet /rta_olap/input/vehicles/

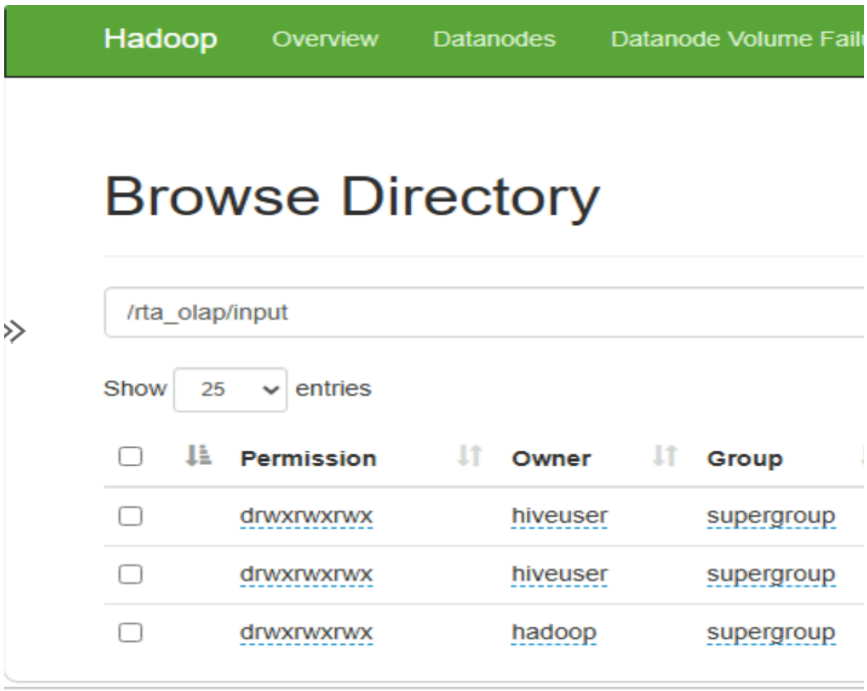
hdfs dfs -put -f /home/hadoop/rta_olap/input/Casualties.parquet /rta_olap/input/casualties/

Verify files uploaded:

hdfs dfs -ls /rta_olap/input/collisions
hdfs dfs -ls /rta_olap/input/vehicles
hdfs dfs -ls /rta_olap/input/casualties

```
hadoop@hallam:~$ hdfs dfs -ls /rta_olap/input/collisions
hdfs dfs -ls /rta_olap/input/vehicles
hdfs dfs -ls /rta_olap/input/casualties
Found 1 items
-rw-r--r--   1 hadoop supergroup    5219720 2026-05-02 16:06 /rta_olap/input/collisions/Collisions.parquet
Found 1 items
-rw-r--r--   1 hadoop supergroup    8549661 2026-05-02 16:06 /rta_olap/input/vehicles/Vehicles.parquet
Found 1 items
-rw-r--r--   1 hadoop supergroup    6254309 2026-05-02 16:07 /rta_olap/input/casualties/Casualties.parquet
hadoop@hallam:~$
```

hdfs dfs -chmod -R 777 /rta_olap **(set permissions for hive to read from hadoop rts_olap folder in the data-lake)**



New Command Line Tab:
ssh hadoop@localhost
Password: Hadoop

Start Hive Metastore service:
hive --service metastore & **('&' to run in background)**

```
Last login: Sat May  2 09:50:57 2026 from 10.0.2.2
hadoop@hallam:~$ hive --service metastore &
[1] 2746
hadoop@hallam:~$ SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/hadoop/apache-hive-4.0.1-bin/lib/log4j-slf4j-impl-2.18.0.jar!/org/slf4j/impl/StaticLoggerBinder.c
lass]
SLF4J: Found binding in [jar:file:/home/hadoop/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLogge
rBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
2026-05-02 10:01:15: Starting Hive Metastore Server
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/hadoop/apache-hive-4.0.1-bin/lib/log4j-slf4j-impl-2.18.0.jar!/org/slf4j/impl/StaticLoggerBinder.c
lass]
SLF4J: Found binding in [jar:file:/home/hadoop/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLogge
rBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
```

New Command Line Tab:
ssh hadoop@localhost
Password: hadoop

Start HiveServer2 service (for Beeline connections)
hive --service hiveserver2 & **('&' to run in background)**

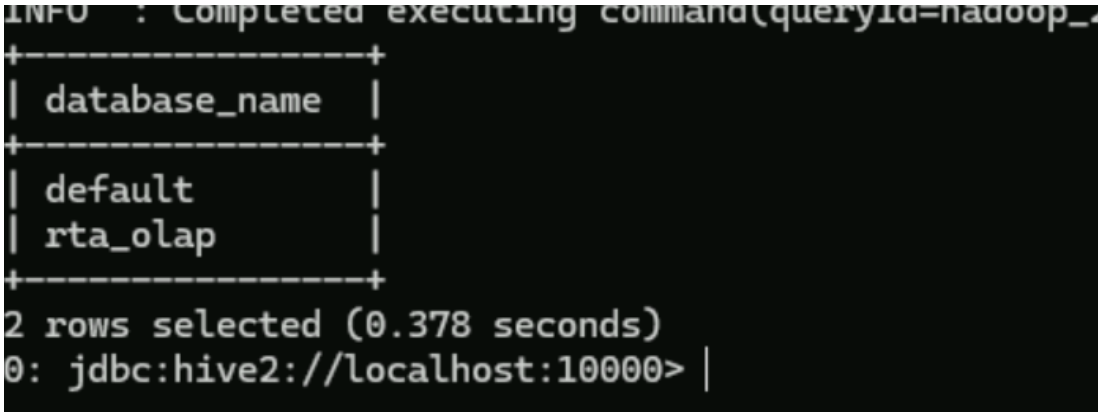
```
Last login: Sat May  2 09:51:14 2026 from 10.0.2.2
hadoop@hallam:~$ hive --service hiveserver2 &
[1] 2915
hadoop@hallam:~$ SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/hadoop/apache-hive-4.0.1-bin/lib/log4j-slf4j-impl-2.18.0.jar!/org/slf4j/impl/StaticLoggerBinder
lass]
SLF4J: Found binding in [jar:file:/home/hadoop/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLog
rBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/hadoop/apache-hive-4.0.1-bin/lib/log4j-slf4j-impl-2.18.0.jar!/org/slf4j/impl/StaticLoggerBinder
lass]
SLF4J: Found binding in [jar:file:/home/hadoop/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLog
rBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
```

New Command Line Tab:
ssh hadoop@localhost
Password: hadoop

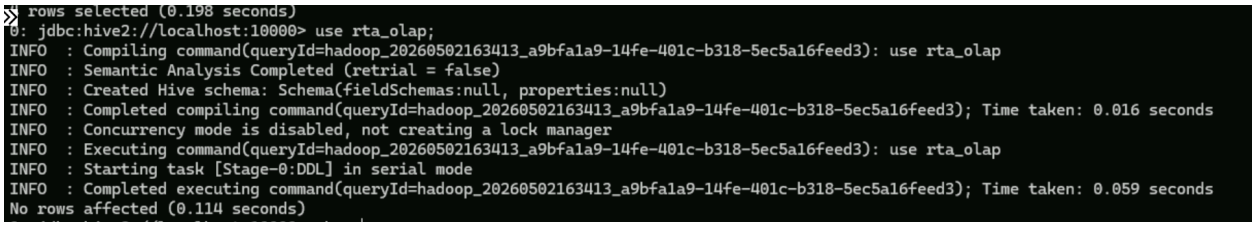
Connect to Hive via Beeline (HiveServer2):
beeline -u jdbc:hive2://localhost:10000 -n hiveuser -p hive

```
hadoop@hallam:~$ beeline -u jdbc:hive2://localhost:10000 -n hiveuser -p hive
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/hadoop/apache-hive-4.0.1-bin/lib/log4j-slf4j-impl-2.18.0.jar!/org/slf4j/impl/StaticLoggerBinder.c
lass]
SLF4J: Found binding in [jar:file:/home/hadoop/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLogge
rBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/hadoop/apache-hive-4.0.1-bin/lib/log4j-slf4j-impl-2.18.0.jar!/org/slf4j/impl/StaticLoggerBinder.c
lass]
SLF4J: Found binding in [jar:file:/home/hadoop/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLogge
rBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Connecting to jdbc:hive2://localhost:10000
Connected to: Apache Hive (version 4.0.1)
Driver: Hive JDBC (version 4.0.1)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 4.0.1 by Apache Hive
0: jdbc:hive2://localhost:10000>
```

Once inside Beeline, you can create a Hive database olap for our project:
CREATE DATABASE rta_olap;



Use the Database created rta_olap to create tables inside:
USE rta_olap;



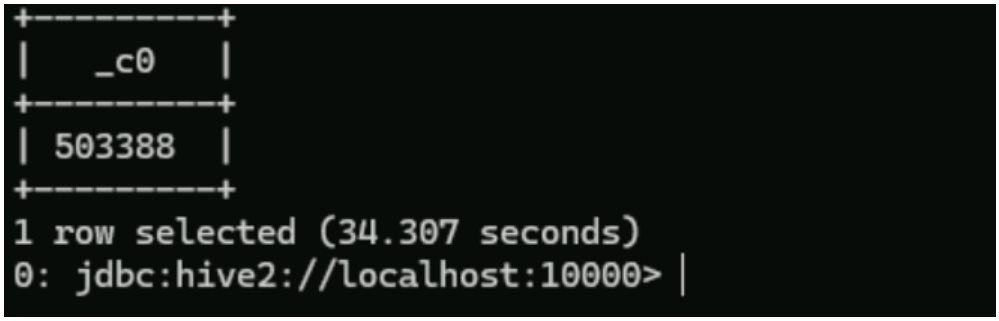
……..

CREATE EXTERNAL TABLE: STAGING TABLES
Here, we will create the staging tables. The staging table is also known as data landing table on hive pointing to our perquet raw data at the Data-lake/ Data Warehouse. This is also known as the external table on hive: collisions, vehicles, casualties.

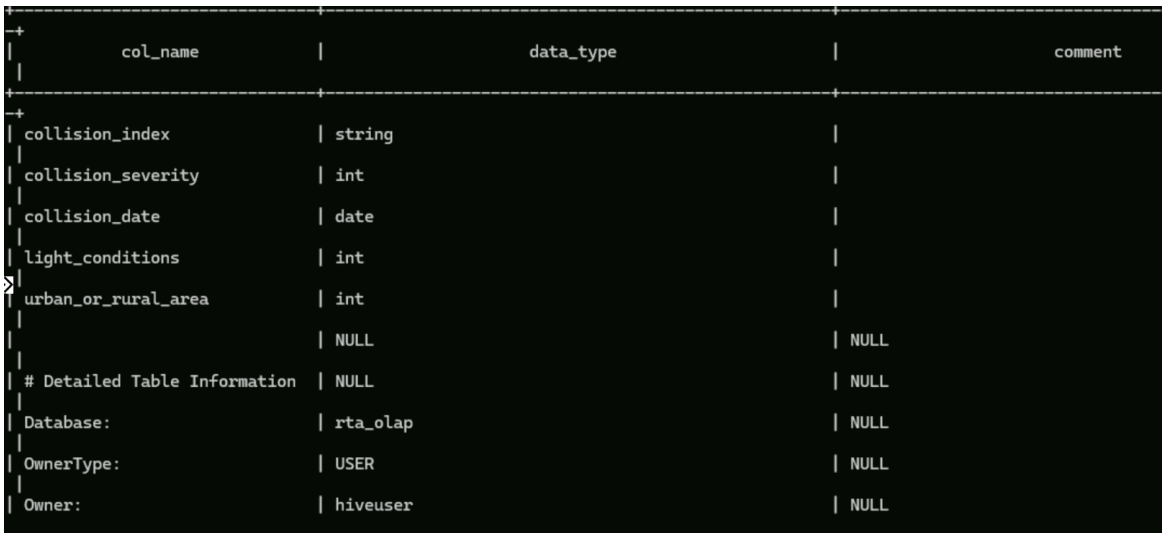
Collisions External Table:

```
CREATE TABLE collisions (
   collision_index STRING,
   collision_severity INT,
   collision_date DATE,
   light_conditions INT,
   urban_or_rural_area INT
)
STORED AS PARQUET
LOCATION '/rta_olap/input/collisions'
TBLPROPERTIES('parquet.compression'='SNAPPY','skip.header.line.count'='1' );
```

Counts the total number of rows (records) in the collisions table:
SELECT COUNT(*) FROM collisions;



Vet Collisions Table Properties:
Describe formatted collisions;



[1] "collision_index" "collision_severity" "collision_date"
[4] "light_conditions" "urban_or_rural_area"

Check Collisions Variables:
Describe collisions;

```
+-----------------------+------------+----------+
|       col_name        | data_type  | comment  |
+-----------------------+------------+----------+
| collision_index       | string     |          |
| collision_severity    | int        |          |
| collision_date        | date       |          |
| light_conditions      | int        |          |
| urban_or_rural_area   | int        |          |
+-----------------------+------------+----------+
5 rows selected (0.193 seconds)
0: jdbc:hive2://localhost:10000>
```

Vet the data if successfully loaded into the collisions external table:
select * from collisions limit 10;

collisions.collision_index	collisions.collision_severity	collisions.collision_date	collisions.light_conditions	collisions.urban_or_rural_area
2021170H10421	3	2021-05-22	6	2
2021170H11231	3	2021-10-20	1	1
2020170M11750	3	2020-12-01	4	1
2021170M31761	3	2021-12-09	4	2
2021170S10441	3	2021-04-12	1	2
2022170S10952	3	2022-07-05	1	1
2021111119419	2	2021-12-09	1	2
2022111155778	3	2022-03-18	1	1
2022111200178	2	2022-06-22	1	1
2020070754597	3	2020-12-12	1	2

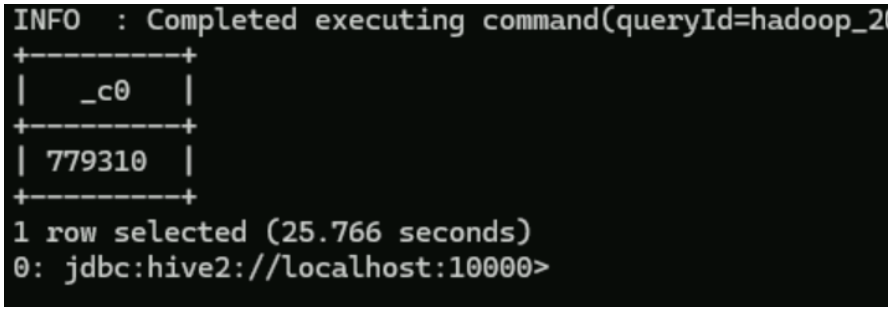
……..

Vehicles External Table:

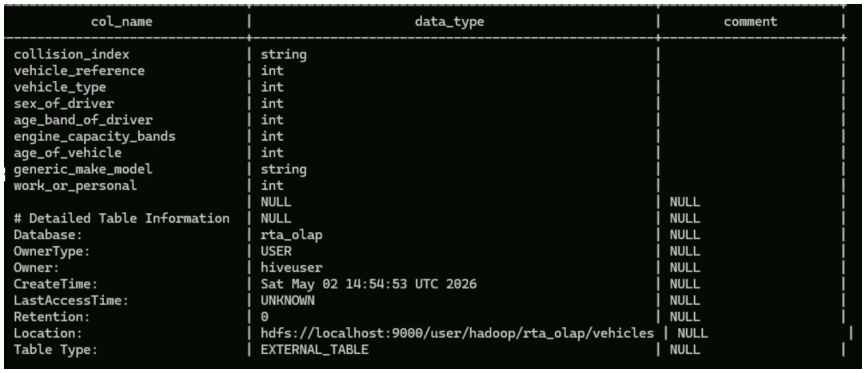
```
CREATE TABLE vehicles (
    collision_index STRING,
    vehicle_reference INT,
    vehicle_type INT,
    sex_of_driver INT,
    age_band_of_driver INT,
    engine_capacity_bands INT,
    age_of_vehicle INT,
    generic_make_model STRING,
    work_or_personal INT
)
STORED AS PARQUET
LOCATION '/user/hadoop/rta_olap/vehicles/'
TBLPROPERTIES (
```

```
    'parquet.compression' = 'SNAPPY',
    'skip.header.line.count' = '1'
);
```

Counts the total number of rows (records) in the vehicles table:
SELECT COUNT(*) FROM vehicles;



Vet Vehicles Table Properties:
Describe formatted vehicles;



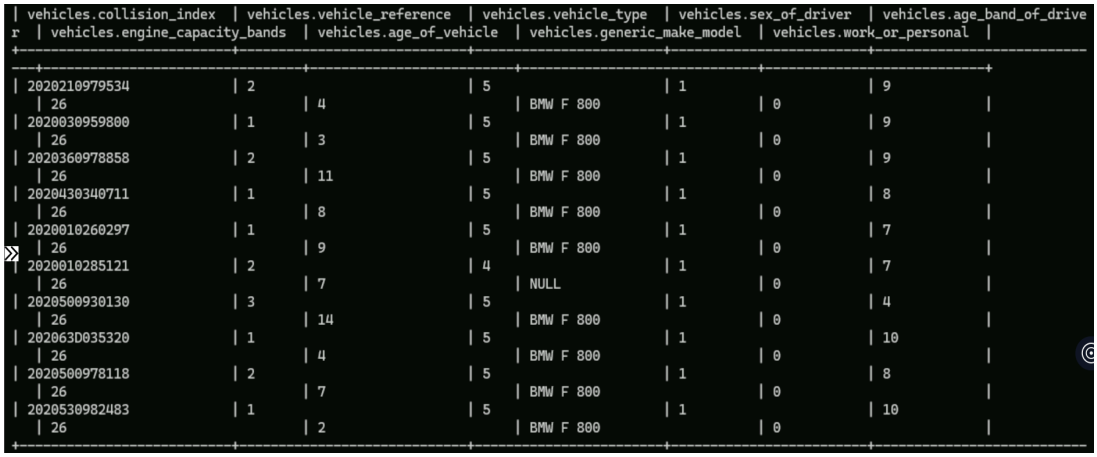
col_name	data_type	comment
collision_index	string	
vehicle_reference	int	
vehicle_type	int	
sex_of_driver	int	
age_band_of_driver	int	
engine_capacity_bands	int	
age_of_vehicle	int	
generic_make_model	string	
work_or_personal	int	
	NULL	NULL
# Detailed Table Information	NULL	NULL
Database:	rta_olap	NULL
OwnerType:	USER	NULL
Owner:	hiveuser	NULL
CreateTime:	Sat May 02 14:54:53 UTC 2026	NULL
LastAccessTime:	UNKNOWN	NULL
Retention:	0	NULL
Location:	hdfs://localhost:9000/user/hadoop/rta_olap/vehicles	NULL
Table Type:	EXTERNAL_TABLE	NULL

[1] "collision_index" "vehicle_reference" "vehicle_type"
[4] "sex_of_driver" "age_band_of_driver" "engine_capacity_bands"
[7] "age_of_vehicle" "generic_make_model" "work_or_personal"

Check Vehicles Variable:
Describe vehicles;

```
+-------------------------+------------+----------+
|        col_name         | data_type  | comment  |
+-------------------------+------------+----------+
| collision_index         | string     |          |
| vehicle_reference       | int        |          |
| vehicle_type            | int        |          |
| sex_of_driver           | int        |          |
| age_band_of_driver      | int        |          |
| engine_capacity_bands   | int        |          |
| age_of_vehicle          | int        |          |
| generic_make_model      | string     |          |
| work_or_personal        | int        |          |
+-------------------------+------------+----------+
9 rows selected (0.192 seconds)
0: jdbc:hive2://localhost:10000>
```

Vet the data if successfully loaded into the vehicles external table:
select * from vehicles limit 10;



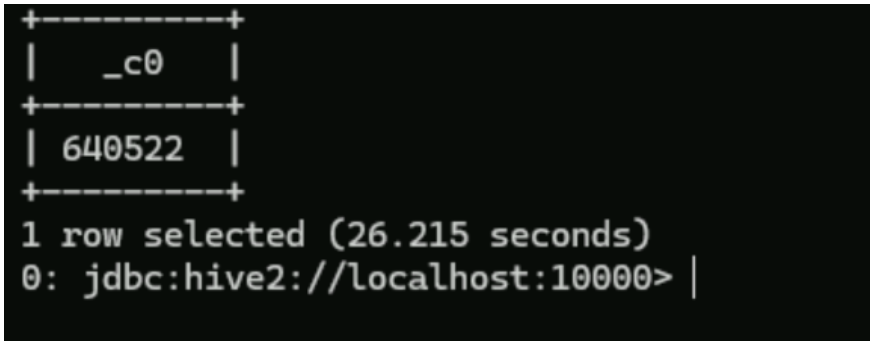
vehicles.collision_index	vehicles.vehicle_reference	vehicles.vehicle_type	vehicles.sex_of_driver	vehicles.age_band_of_driver	vehicles.engine_capacity_bands	vehicles.age_of_vehicle	vehicles.generic_make_model	vehicles.work_or_personal
2020210979534	2	5	1	9	26	4	BMW F 800	0
2020030959800	1	5	1	9	26	3	BMW F 800	0
2020360978858	2	5	1	9	26	11	BMW F 800	0
2020430340711	1	5	1	8	26	8	BMW F 800	0
2020010260297	1	5	1	7	26	9	BMW F 800	0
2020010285121	2	4	1	7	26	7	NULL	0
2020500930130	3	5	1	4	26	14	BMW F 800	0
202063D035320	1	5	1	10	26	4	BMW F 800	0
2020500978118	2	5	1	8	26	7	BMW F 800	0
2020530982483	1	5	1	10	26	2	BMW F 800	0

……

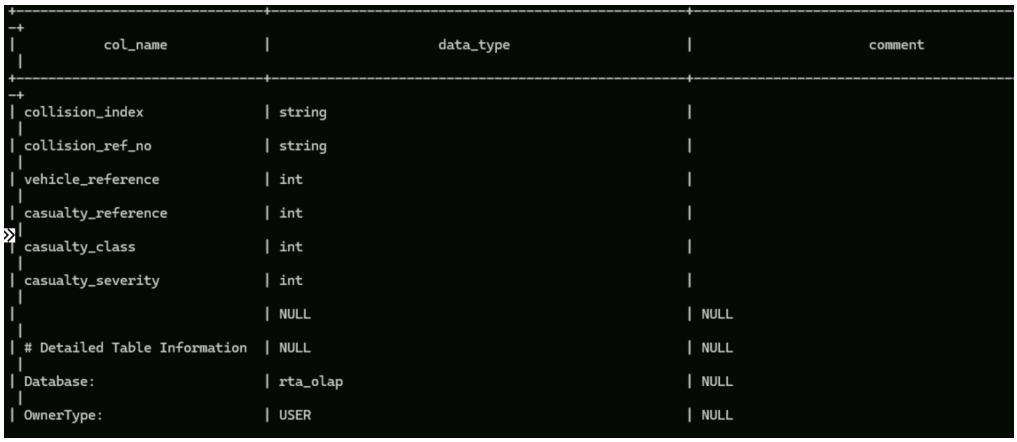
Casualties External Table:

```
CREATE TABLE casualties (
    collision_index STRING,
    vehicle_reference INT,
    casualty_reference INT,
    casualty_severity INT
)
STORED AS PARQUET
LOCATION '/rta_olap/input/casualties'
TBLPROPERTIES (
    'parquet.compression' = 'SNAPPY',
    'skip.header.line.count' = '1'
);
```

Counts the total number of rows (records) in the casualties table:
SELECT COUNT(*) FROM casualties;



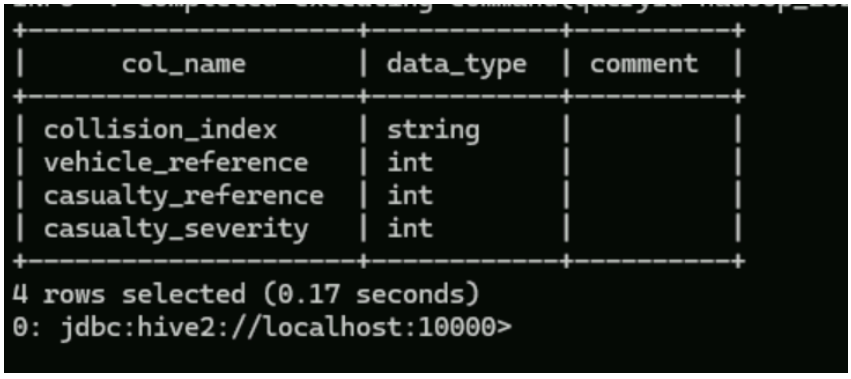
Vet Casualties Table Property:
Describe formatted casualties;



col_name	data_type	comment
collision_index	string	
collision_ref_no	string	
vehicle_reference	int	
casualty_reference	int	
casualty_class	int	
casualty_severity	int	
	NULL	NULL
# Detailed Table Information	NULL	NULL
Database:	rta_olap	NULL
OwnerType:	USER	NULL

[1] "collision_index" "vehicle_reference" "casualty_reference" [4] "casualty_severity"

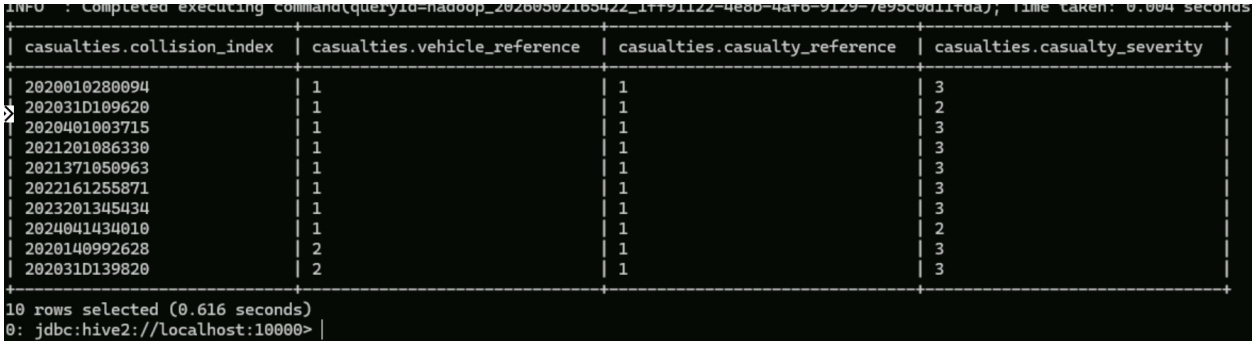
Check Collions Variables:
Describe collisions;



col_name	data_type	comment
collision_index	string	
vehicle_reference	int	
casualty_reference	int	
casualty_severity	int	

4 rows selected (0.17 seconds)
0: jdbc:hive2://localhost:10000>

Vet the data if successfully loaded into the casualties external table:
select * from casualties limit 10;



……..

DIM_TIME_TEMP AND DIM_TIME TABLE:

The **dim_time_temp** table is a staging table used to extract and transform time-related data from the raw dataset. The cleaned data is then loaded into the **dim_time** table, which serves as the final dimension table used for analysis in the star schema.

Create Dim_time_temp:

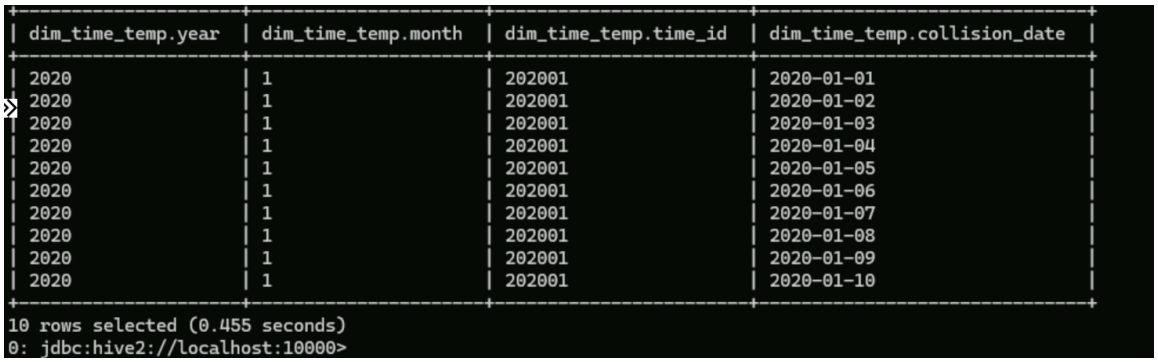
Purpose: keep raw detailed dates of events not useful for our analysis

```
CREATE TABLE dim_time_temp (
   year INT,
   month INT,
   time_id INT,
   collision_date DATE
)
STORED AS PARQUET
TBLPROPERTIES (
   'parquet.compression'='SNAPPY',
   'skip.header.line.count'='1'
);
```

```
INSERT TABLE dim_time_temp
SELECT DISTINCT
   YEAR(CAST(collision_date AS DATE)),
   MONTH(CAST(collision_date AS DATE)),
   CAST(CONCAT(
      YEAR(CAST(collision_date AS DATE)),
      LPAD(MONTH(CAST(collision_date AS DATE)), 2, '0')
   ) AS INT),
   CAST(collision_date AS DATE)
```

FROM collisions
WHERE collision_date IS NOT NULL; **(we don't want null column to load. We want recorded date be loaded into our temporary time dimension table)**

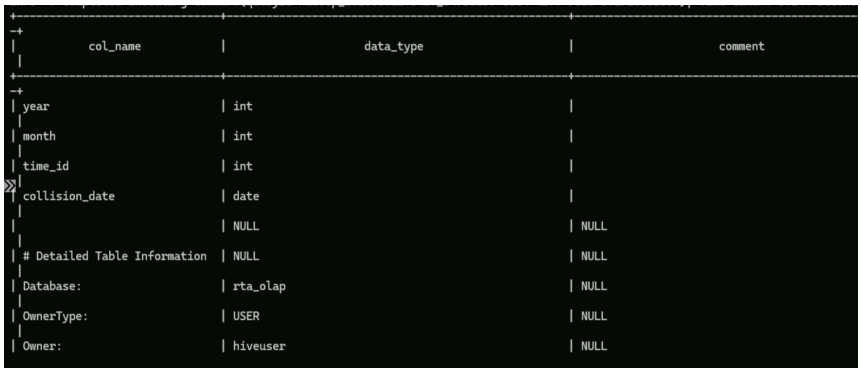
Vet dim_time_temp table if there's no null:
Select * from dim_time_temp;



dim_time_temp.year	dim_time_temp.month	dim_time_temp.time_id	dim_time_temp.collision_date
2020	1	202001	2020-01-01
2020	1	202001	2020-01-02
2020	1	202001	2020-01-03
2020	1	202001	2020-01-04
2020	1	202001	2020-01-05
2020	1	202001	2020-01-06
2020	1	202001	2020-01-07
2020	1	202001	2020-01-08
2020	1	202001	2020-01-09
2020	1	202001	2020-01-10

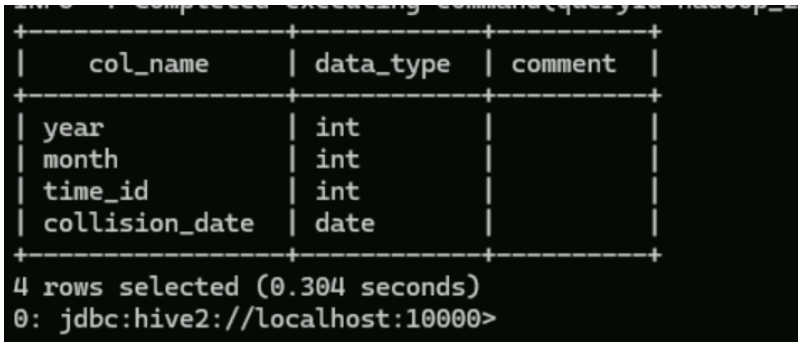
10 rows selected (0.455 seconds)
0: jdbc:hive2://localhost:10000>

Vet dim_time_table properties:
Describe formatted dim_time_temp;



col_name	data_type	comment
year	int	
month	int	
time_id	int	
collision_date	date	
	NULL	NULL
# Detailed Table Information	NULL	NULL
Database:	rta_olap	NULL
OwnerType:	USER	NULL
Owner:	hiveuser	NULL

Check the variables:
Describe dim_time_temp;



col_name	data_type	comment
year	int	
month	int	
time_id	int	
collision_date	date	

4 rows selected (0.304 seconds)
0: jdbc:hive2://localhost:10000>

……

Create Dim_time:

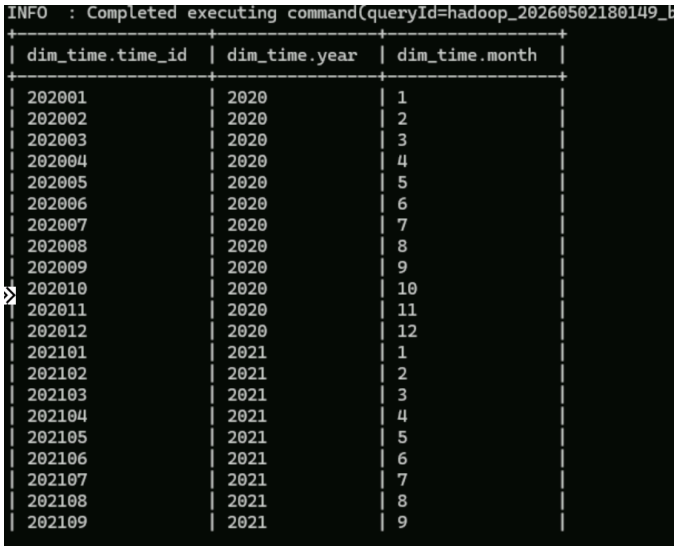
This is our permanent dimension time table for our analysis. It defines lowest level granularity: month and year. In our analysis, we will always refer to this.

Create Table dim_time :

```
CREATE TABLE dim_time (
    time_id INT,
    year INT,
    month INT,
    CONSTRAINT pk_time PRIMARY KEY (time_id) DISABLE NOVALIDATE RELY
)
STORED AS PARQUET
TBLPROPERTIES (
    'parquet.compression'='SNAPPY',
    'skip.header.line.count'='1'
);

INSERT OVERWRITE TABLE dim_time
SELECT DISTINCT
    time_id,
    year,
    month
FROM dim_time_temp;
```

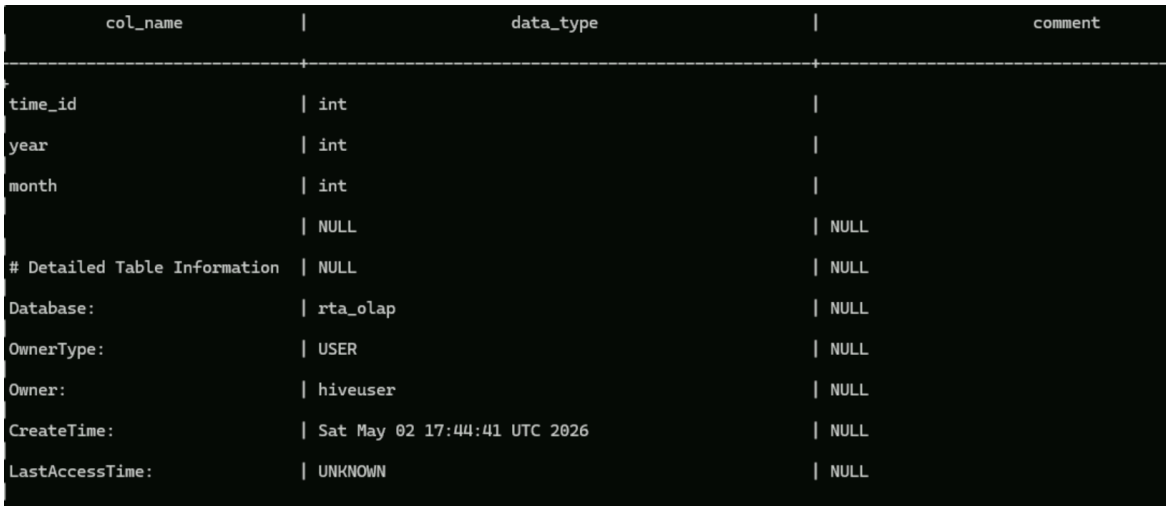
Vet Dim_Time Table created has no Null neither empty:



```
INFO  : Completed executing command(queryId=hadoop_20260502180149_
+-------------------+----------------+-----------------+
| dim_time.time_id  | dim_time.year  | dim_time.month  |
+-------------------+----------------+-----------------+
| 202001            | 2020           | 1               |
| 202002            | 2020           | 2               |
| 202003            | 2020           | 3               |
| 202004            | 2020           | 4               |
| 202005            | 2020           | 5               |
| 202006            | 2020           | 6               |
| 202007            | 2020           | 7               |
| 202008            | 2020           | 8               |
| 202009            | 2020           | 9               |
| 202010            | 2020           | 10              |
| 202011            | 2020           | 11              |
| 202012            | 2020           | 12              |
| 202101            | 2021           | 1               |
| 202102            | 2021           | 2               |
| 202103            | 2021           | 3               |
| 202104            | 2021           | 4               |
| 202105            | 2021           | 5               |
| 202106            | 2021           | 6               |
| 202107            | 2021           | 7               |
| 202108            | 2021           | 8               |
| 202109            | 2021           | 9               |
```

Vet Dim_Time Properties:

Describe formatted dim_time



col_name	data_type	comment
time_id	int	
year	int	
month	int	
	NULL	NULL
# Detailed Table Information	NULL	NULL
Database:	rta_olap	NULL
OwnerType:	USER	NULL
Owner:	hiveuser	NULL
CreateTime:	Sat May 02 17:44:41 UTC 2026	NULL
LastAccessTime:	UNKNOWN	NULL

Check dim_time variables:
Describe dim_time;



col_name	data_type	comment
time_id	int	
year	int	
month	int	

3 rows selected (0.17 seconds)
0: jdbc:hive2://localhost:10000>

.....

DIMENSION TABLES:

Dimension tables are created to store descriptive information that gives context to the data in fact tables. They are known with qualitative variables and qualitative data. Data with labels.

Creating Dimension Tables for each of our business questions:

CREATE TABLE dim_condition (
  condition_id INT,
  light_conditions INT,
  CONSTRAINT pk_dim_condition PRIMARY KEY (condition_id) DISABLE NOVALIDATE RELY
)
STORED AS PARQUET
TBLPROPERTIES (

```
    'parquet.compression' = 'SNAPPY',
    'skip.header.line.count' = '1'
);

INSERT INTO TABLE dim_condition
SELECT DISTINCT
light_conditions AS condition_id,
light_conditions
FROM collisions;
```

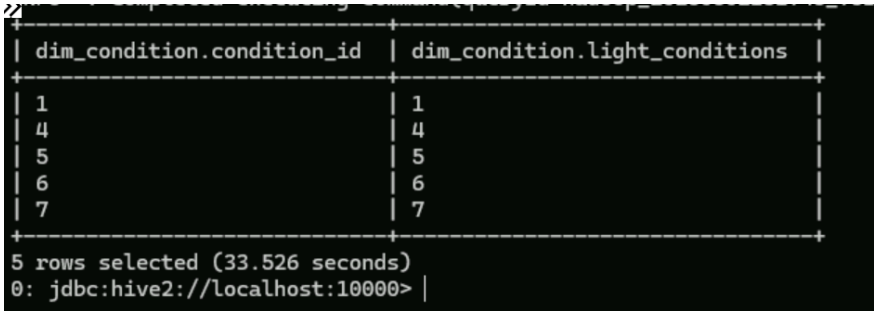
Check dim_condition variables:

Describe dim_condition;



```
+--------------------+------------+----------+
|      col_name      | data_type  | comment  |
+--------------------+------------+----------+
| condition_id       | int        |          |
| light_conditions   | int        |          |
+--------------------+------------+----------+
2 rows selected (0.168 seconds)
0: jdbc:hive2://localhost:10000>
```

Select * from dim_condition limit 10;



```
+-----------------------------+---------------------------------+
| dim_condition.condition_id  | dim_condition.light_conditions  |
+-----------------------------+---------------------------------+
| 1                           | 1                               |
| 4                           | 4                               |
| 5                           | 5                               |
| 6                           | 6                               |
| 7                           | 7                               |
+-----------------------------+---------------------------------+
5 rows selected (33.526 seconds)
0: jdbc:hive2://localhost:10000>
```

……

```
CREATE TABLE dim_area (
    area_id INT,
    urban_or_rural_area INT,
    CONSTRAINT pk_dim_area PRIMARY KEY (area_id) DISABLE NOVALIDATE RELY
)
STORED AS PARQUET
TBLPROPERTIES (
    'parquet.compression'='SNAPPY',
    'skip.header.line.count'='1'
);

INSERT INTO TABLE dim_area
SELECT DISTINCT
```

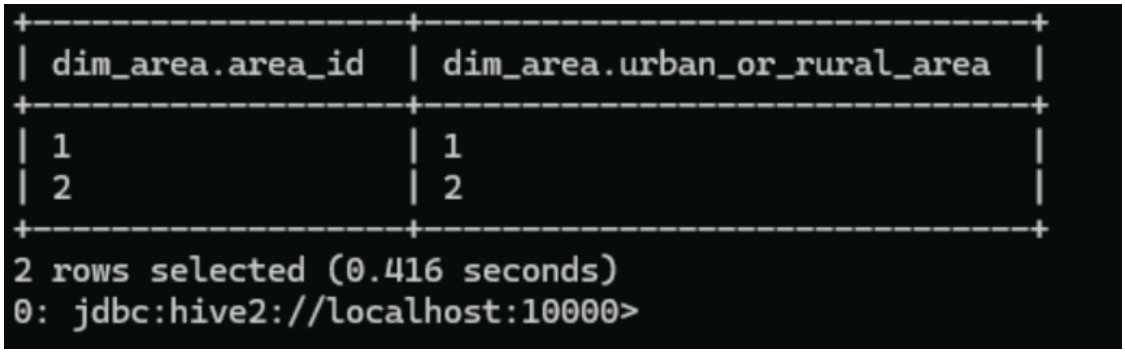
```
    urban_or_rural_area AS area_id,
    urban_or_rural_area
FROM collisions;
```

Check dim_area variables:
Describe dim_area;



```
+---------------------+-----------+----------+
|       col_name      | data_type | comment  |
+---------------------+-----------+----------+
| area_id             | int       |          |
| urban_or_rural_area | int       |          |
+---------------------+-----------+----------+
2 rows selected (0.18 seconds)
0: jdbc:hive2://localhost:10000>
```

Vet the table:
Select * from dim_area limit 10;



```
+------------------+-------------------------------+
| dim_area.area_id | dim_area.urban_or_rural_area  |
+------------------+-------------------------------+
| 1                | 1                             |
| 2                | 2                             |
+------------------+-------------------------------+
2 rows selected (0.416 seconds)
0: jdbc:hive2://localhost:10000>
```

……..

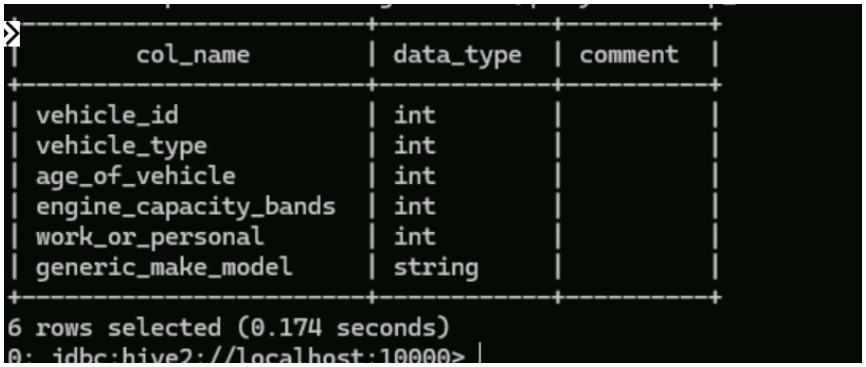
```
CREATE TABLE dim_vehicle (
    vehicle_id INT,
    vehicle_type INT,
    age_of_vehicle INT,
    engine_capacity_bands INT,
    work_or_personal INT,
    generic_make_model STRING,
    CONSTRAINT pk_vehicle PRIMARY KEY (vehicle_id) DISABLE NOVALIDATE RELY
)
STORED AS PARQUET
TBLPROPERTIES (
    'parquet.compression'='SNAPPY',
    'skip.header.line.count'='1'
);
```

INSERT INTO TABLE dim_vehicle

```
SELECT DISTINCT
   DENSE_RANK() OVER (
      ORDER BY vehicle_type, age_of_vehicle, engine_capacity_bands,
            work_or_personal, generic_make_model
   ) AS vehicle_id,
   vehicle_type,
   age_of_vehicle,
   engine_capacity_bands,
   work_or_personal,
   generic_make_model
FROM vehicles;
```

Check dim_vehicle variables:

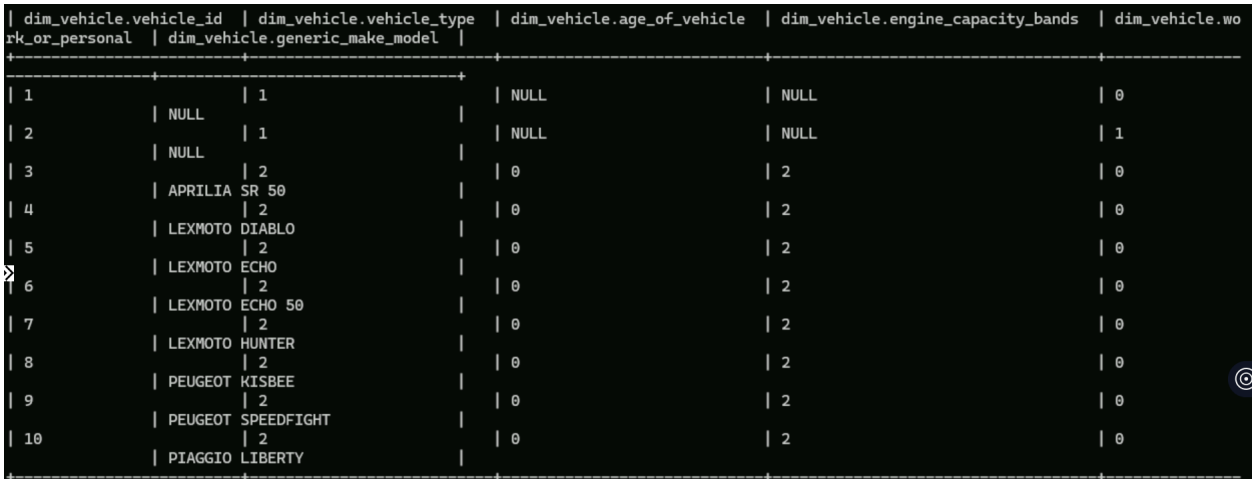
Describe dim_vehicle;



```
+-------------------------+------------+----------+
|        col_name         | data_type  | comment  |
+-------------------------+------------+----------+
| vehicle_id              | int        |          |
| vehicle_type            | int        |          |
| age_of_vehicle          | int        |          |
| engine_capacity_bands   | int        |          |
| work_or_personal        | int        |          |
| generic_make_model      | string     |          |
+-------------------------+------------+----------+
6 rows selected (0.174 seconds)
0: jdbc:hive2://localhost:10000>
```

Vet the table:

Select * from dim_vehicle limit 10;

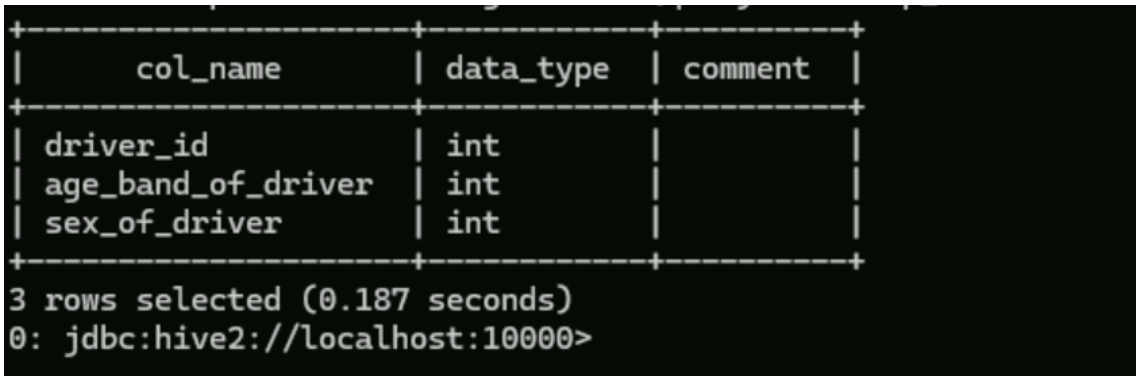


```
| dim_vehicle.vehicle_id  | dim_vehicle.vehicle_type  | dim_vehicle.age_of_vehicle  | dim_vehicle.engine_capacity_bands  | dim_vehicle.wo
rk_or_personal  | dim_vehicle.generic_make_model  |
+-------------------------+---------------------------+-----------------------------+------------------------------------+---------------
----------------+---------------------------------+
| 1                       | 1                         | NULL                        | NULL                               | 0
                | NULL                            |
| 2                       | 1                         | NULL                        | NULL                               | 1
                | NULL                            |
| 3                       | 2                         | 0                           | 2                                  | 0
                | APRILIA SR 50                   |
| 4                       | 2                         | 0                           | 2                                  | 0
                | LEXMOTO DIABLO                  |
| 5                       | 2                         | 0                           | 2                                  | 0
                | LEXMOTO ECHO                    |
| 6                       | 2                         | 0                           | 2                                  | 0
                | LEXMOTO ECHO 50                 |
| 7                       | 2                         | 0                           | 2                                  | 0
                | LEXMOTO HUNTER                  |
| 8                       | 2                         | 0                           | 2                                  | 0
                | PEUGEOT KISBEE                  |
| 9                       | 2                         | 0                           | 2                                  | 0
                | PEUGEOT SPEEDFIGHT              |
| 10                      | 2                         | 0                           | 2                                  | 0
                | PIAGGIO LIBERTY                 |
+-------------------------+---------------------------+-----------------------------+------------------------------------+---------------
----------------+---------------------------------+
```

CREATE TABLE dim_driver (
  driver_id INT,
  age_band_of_driver INT,
  sex_of_driver INT,
  CONSTRAINT pk_dim_driver PRIMARY KEY (driver_id) DISABLE NOVALIDATE RELY
)
STORED AS PARQUET
TBLPROPERTIES('parquet.compression'='SNAPPY','skip.header.line.count'='1');

INSERT INTO TABLE dim_driver
SELECT DISTINCT
  DENSE_RANK() OVER (ORDER BY age_band_of_driver, sex_of_driver) AS driver_id,
  age_band_of_driver,
  sex_of_driver
FROM vehicles;

Check for dim_driver variables:
describe dim_driver;



Vet the table:
Select * from dim_driver limit 10;

```
INFO  : Completed executing command(queryId=hadoop_20260502190352_d9c222ea-4cde-4df5-abc0-
+-------------------------+-----------------------------------+---------------------------+
| dim_driver.driver_id    | dim_driver.age_band_of_driver     | dim_driver.sex_of_driver  |
+-------------------------+-----------------------------------+---------------------------+
| 1                       | 1                                 | 1                         |
| 2                       | 1                                 | 2                         |
| 3                       | 1                                 | 3                         |
| 4                       | 2                                 | 1                         |
| 5                       | 2                                 | 2                         |
| 6                       | 2                                 | 3                         |
| 7                       | 3                                 | 1                         |
| 8                       | 3                                 | 2                         |
| 9                       | 3                                 | 3                         |
| 10                      | 3                                 | NULL                      |
+-------------------------+-----------------------------------+---------------------------+
10 rows selected (1.796 seconds)
0: jdbc:hive2://localhost:10000>
```

……..

CREATE TABLE dim_severity (
  severity_id INT,
  collision_severity INT,
  CONSTRAINT pk_dim_severity PRIMARY KEY (severity_id) DISABLE NOVALIDATE RELY
)
STORED AS PARQUET
TBLPROPERTIES('parquet.compression'='SNAPPY','skip.header.line.count'='1');

INSERT TABLE dim_severity
SELECT DISTINCT
  collision_severity AS severity_id,
  collision_severity
FROM collisions;

```
+--------------------------+-----------------------------------+
| dim_severity.severity_id | dim_severity.collision_severity   |
+--------------------------+-----------------------------------+
| 1                        | 1                                 |
| 2                        | 2                                 |
| 3                        | 3                                 |
+--------------------------+-----------------------------------+
3 rows selected (0.406 seconds)
0: jdbc:hive2://localhost:10000>
```

……..

FACT TABLES:

Fact tables store numerical data, like counts of collisions, along with keys that link to dimension tables. They are used to analyse the data by combining these measures with descriptive information such as time, location, vehicle details, urban and rural, and etc.

CREATE FACT TABLES LINKING TO DIMENSION TABLES:

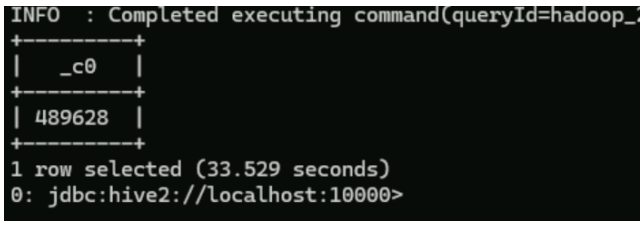
```
CREATE TABLE fct_table_1 (
    time_id INT,
    area_id INT,
    condition_id INT,
    vehicle_id INT,
    severity_id INT,
    severity_count INT
)
STORED AS PARQUET
TBLPROPERTIES (
    'parquet.compression'='SNAPPY',
    'skip.header.line.count'='1'
);
```

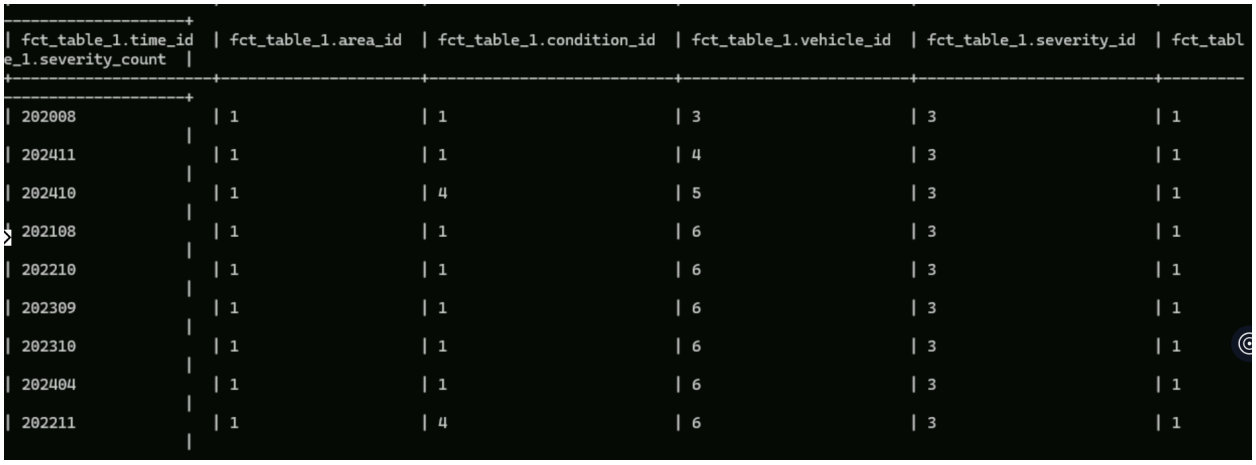
```
INSERT INTO TABLE fct_table_1
SELECT
    t.time_id,
    dim_a.area_id,
    dim_c.condition_id,
    dim_v.vehicle_id,
    dim_s.severity_id,
    COUNT(*) AS severity_count
FROM collisions co
JOIN vehicles v
    ON v.collision_index = co.collision_index
JOIN dim_time_temp t
    ON co.collision_date = t.collision_date
JOIN dim_area dim_a
    ON co.urban_or_rural_area = dim_a.urban_or_rural_area
JOIN dim_condition dim_c
    ON co.light_conditions = dim_c.light_conditions
JOIN dim_vehicle dim_v
    ON v.vehicle_type = dim_v.vehicle_type
   AND v.age_of_vehicle = dim_v.age_of_vehicle
   AND (
        v.engine_capacity_bands = dim_v.engine_capacity_bands
```

```
      OR (v.engine_capacity_bands IS NULL AND dim_v.engine_capacity_bands IS NULL)
  )
  AND v.work_or_personal = dim_v.work_or_personal
  AND (
      v.generic_make_model = dim_v.generic_make_model
      OR (v.generic_make_model IS NULL AND dim_v.generic_make_model IS NULL)
  )
JOIN dim_severity dim_s
   ON co.collision_severity = dim_s.collision_severity
GROUP BY
   t.time_id,
   dim_a.area_id,
   dim_c.condition_id,
   dim_v.vehicle_id,
   dim_s.severity_id;
```

What is count all?
COUNT(*) = count all rows in the table:
After creating or inserting into a table, we want to know:
Did data load successfully?
Is the table empty?
How big is the dataset?

```
SELECT COUNT(*) FROM fct_table_1;
SELECT * FROM fct_table_1 LIMIT 10;
```





fct_table_1.time_id	fct_table_1.area_id	fct_table_1.condition_id	fct_table_1.vehicle_id	fct_table_1.severity_id	fct_table_1.severity_count
202008	1	1	3	3	1
202411	1	1	4	3	1
202410	1	4	5	3	1
202108	1	1	6	3	1
202210	1	1	6	3	1
202309	1	1	6	3	1
202310	1	1	6	3	1
202404	1	1	6	3	1
202211	1	4	6	3	1

.....

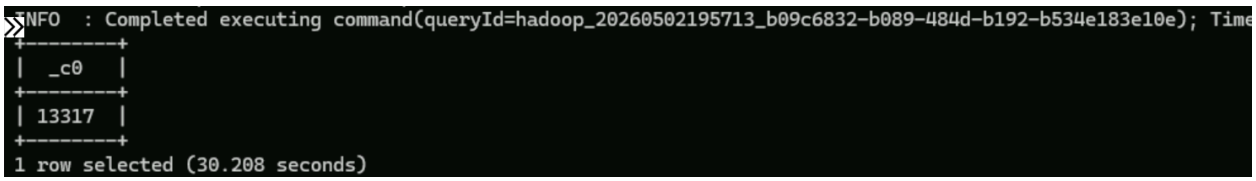
```
CREATE TABLE fct_table_2 (
    time_id INT,
    driver_id INT,
    condition_id INT,
    severity_id INT,
    severity_count INT
)
STORED AS PARQUET
TBLPROPERTIES (
    'parquet.compression'='SNAPPY',
    'skip.header.line.count'='1'
);

INSERT INTO TABLE fct_table_2
SELECT
    t.time_id,
    dim_d.driver_id,
    dim_c.condition_id,
    dim_s.severity_id,
    COUNT(*) AS severity_count
FROM collisions
JOIN vehicles v
    ON v.collision_index = co.collision_index
JOIN dim_time_temp t
    ON co.collision_date = t.collision_date
JOIN dim_driver dim_d
    ON v.age_band_of_driver = dim_d.age_band_of_driver
   AND v.sex_of_driver = dim_d.sex_of_driver
JOIN dim_condition dim_c
```

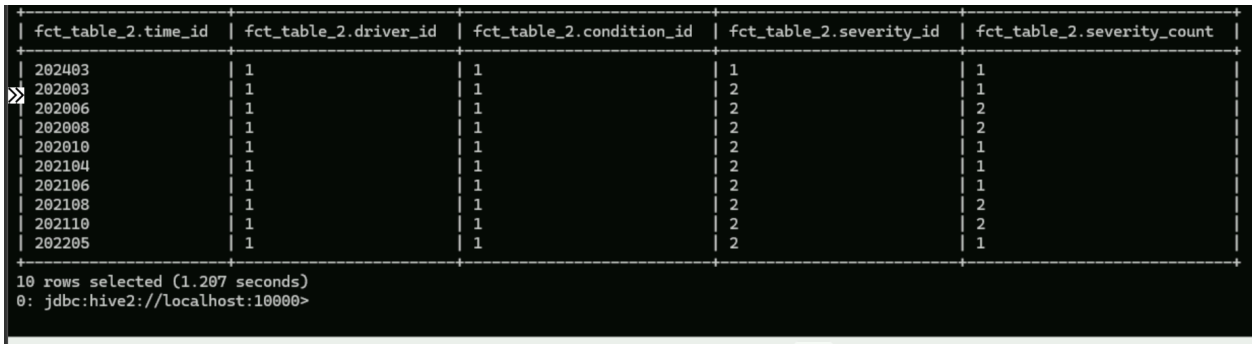
```
   ON co.light_conditions = dim_c.light_conditions
JOIN dim_severity dim_s
   ON co.collision_severity = dim_s.collision_severity
GROUP BY
   t.time_id,
   dim_d.driver_id,
   dim_c.condition_id,
   dim_s.severity_id;
```

Vet fact_2 table:

```
SELECT COUNT(*) FROM fct_table_2;
SELECT * FROM fct_table_2 LIMIT 10;
```



```
INFO  : Completed executing command(queryId=hadoop_20260502195713_b09c6832-b089-484d-b192-b534e183e10e); Time
+--------+
|  _c0   |
+--------+
| 13317  |
+--------+
1 row selected (30.208 seconds)
```



```
+-----------------------+-------------------------+----------------------------+--------------------------+-----------------------------+
| fct_table_2.time_id   | fct_table_2.driver_id   | fct_table_2.condition_id   | fct_table_2.severity_id  | fct_table_2.severity_count  |
+-----------------------+-------------------------+----------------------------+--------------------------+-----------------------------+
| 202403                | 1                       | 1                          | 1                        | 1                           |
| 202003                | 1                       | 1                          | 2                        | 1                           |
| 202006                | 1                       | 1                          | 2                        | 2                           |
| 202008                | 1                       | 1                          | 2                        | 2                           |
| 202010                | 1                       | 1                          | 2                        | 1                           |
| 202104                | 1                       | 1                          | 2                        | 1                           |
| 202106                | 1                       | 1                          | 2                        | 1                           |
| 202108                | 1                       | 1                          | 2                        | 2                           |
| 202110                | 1                       | 1                          | 2                        | 2                           |
| 202205                | 1                       | 1                          | 2                        | 1                           |
+-----------------------+-------------------------+----------------------------+--------------------------+-----------------------------+
10 rows selected (1.207 seconds)
0: jdbc:hive2://localhost:10000>
```

.....

```
CREATE TABLE fct_table_3 (
   time_id INT,
   area_id INT,
   vehicle_id INT,
   severity_id INT,
   severity_count INT
)
STORED AS PARQUET
TBLPROPERTIES (
   'parquet.compression' = 'SNAPPY',
   'skip.header.line.count' = '1'
);
```

```
INSERT INTO TABLE fct_table_3
SELECT
  t.time_id,
  dim_a.area_id,
  dim_v.vehicle_id,
  dim_s.severity_id,
  COUNT(*) AS severity_count
FROM collisions co
JOIN vehicles v
  ON v.collision_index = co.collision_index
JOIN dim_time_temp t
  ON co.collision_date = t.collision_date
JOIN dim_severity dim_s
  ON co.collision_severity = dim_s.collision_severity
JOIN dim_area dim_a
  ON co.urban_or_rural_area = dim_a.urban_or_rural_area
JOIN dim_vehicle dim_v
  ON v.vehicle_type = dim_v.vehicle_type
 AND v.age_of_vehicle = dim_v.age_of_vehicle
 AND (
     v.engine_capacity_bands = dim_v.engine_capacity_bands
     OR (v.engine_capacity_bands IS NULL AND dim_v.engine_capacity_bands IS NULL)
 )
 AND v.work_or_personal = dim_v.work_or_personal
 AND (
     v.generic_make_model = dim_v.generic_make_model
     OR (v.generic_make_model IS NULL AND dim_v.generic_make_model IS NULL)
 )
GROUP BY
  t.time_id,
  dim_a.area_id,
  dim_v.vehicle_id,
  dim_s.severity_id;
```

Vet fact_3 table:

```
SELECT COUNT(*) FROM fct_table_3;
SELECT * FROM fct_table_3 LIMIT 10;
```

```
INFO  : Completed executing command(queryId=hadoop_20260
+----------+
|   _c0    |
+----------+
| 419277   |
+----------+
1 row selected (31.274 seconds)
0: jdbc:hive2://localhost:10000>
```

fct_table_3.time_id	fct_table_3.area_id	fct_table_3.vehicle_id	fct_table_3.severity_id	fct_table_3.severity_count
202008	1	3	3	1
202411	1	4	3	1
202410	1	5	3	1
202108	1	6	3	1
202210	1	6	3	1
202211	1	6	3	1
202309	1	6	3	1
202310	1	6	3	1
202404	1	6	3	1
202309	2	6	2	1

……..

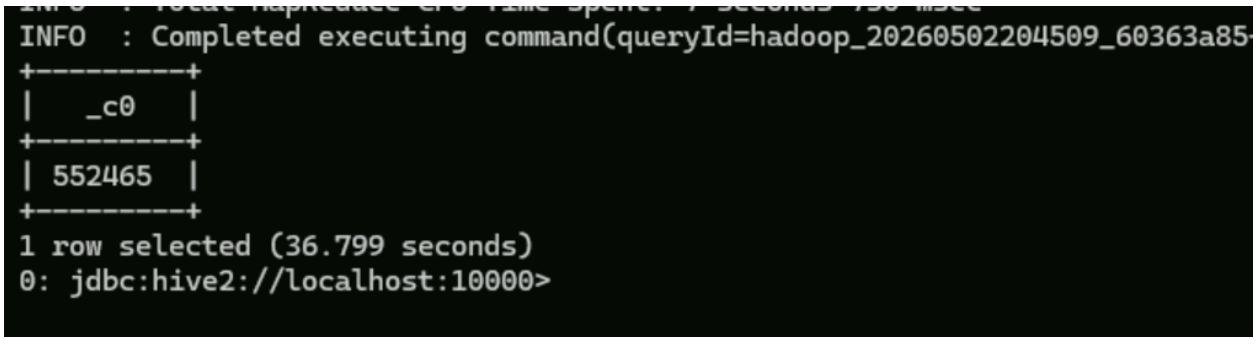
```
CREATE TABLE fct_table_4 (
    time_id INT,
    condition_id INT,
    vehicle_id INT,
    severity_id INT,
    area_id INT,
    severity_count INT
)
STORED AS PARQUET
TBLPROPERTIES (
    'parquet.compression'='SNAPPY',
    'skip.header.line.count'='1'
);

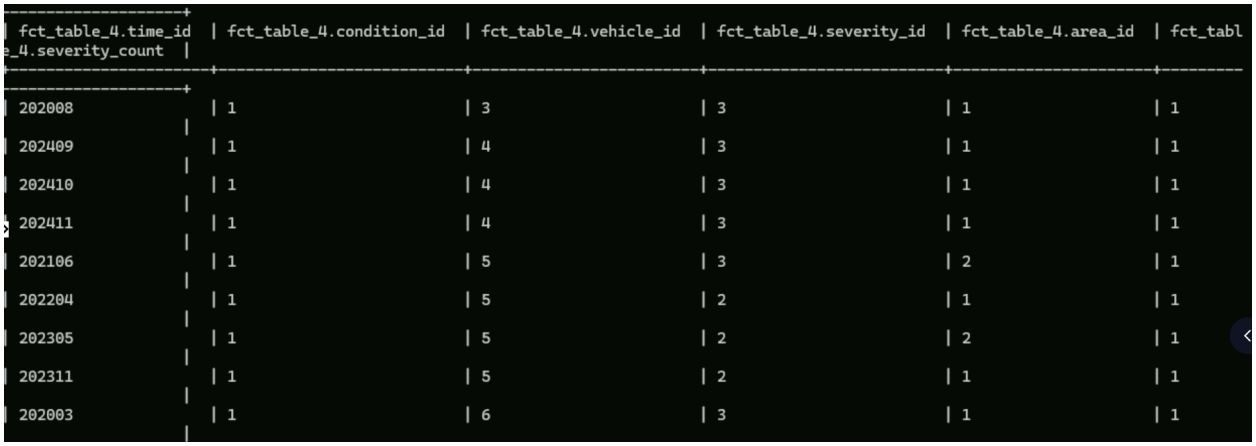
INSERT INTO TABLE fct_table_4
SELECT
    t.time_id,
    dim_c.condition_id,
    dim_v.vehicle_id,
    dim_s.severity_id,
    dim_a.area_id,
    COUNT(*) AS severity_count
FROM collisions co
JOIN vehicles v
    ON v.collision_index = co.collision_index
```

```
JOIN dim_time_temp t
   ON co.collision_date = t.collision_date
JOIN dim_condition dim_c
   ON co.light_conditions = dim_c.light_conditions
JOIN dim_severity dim_s
   ON co.collision_severity = dim_s.collision_severity
JOIN dim_vehicle dim_v
   ON v.vehicle_type = dim_v.vehicle_type
  AND v.age_of_vehicle = dim_v.age_of_vehicle
  AND (
      v.engine_capacity_bands = dim_v.engine_capacity_bands
      OR (v.engine_capacity_bands IS NULL AND dim_v.engine_capacity_bands IS NULL)
  )
  AND v.work_or_personal = dim_v.work_or_personal
  AND (
      v.generic_make_model = dim_v.generic_make_model
      OR (v.generic_make_model IS NULL AND dim_v.generic_make_model IS NULL)
  )
JOIN dim_area dim_a
   ON co.urban_or_rural_area = dim_a.urban_or_rural_area
GROUP BY
   t.time_id,
   dim_c.condition_id,
   dim_v.vehicle_id,
   dim_s.severity_id,
   Dim_a.area_id;
```

Vet fact_4 table:

```
SELECT COUNT(*) FROM fct_table_4;
SELECT * FROM fct_table_4 LIMIT 10;
```





fct_table_4.time_id	fct_table_4.condition_id	fct_table_4.vehicle_id	fct_table_4.severity_id	fct_table_4.area_id	fct_table_4.severity_count
202008	1	3	3	1	1
202409	1	4	3	1	1
202410	1	4	3	1	1
202411	1	4	3	1	1
202106	1	5	3	2	1
202204	1	5	2	1	1
202305	1	5	2	2	1
202311	1	5	2	1	1
202003	1	6	3	1	1

.....

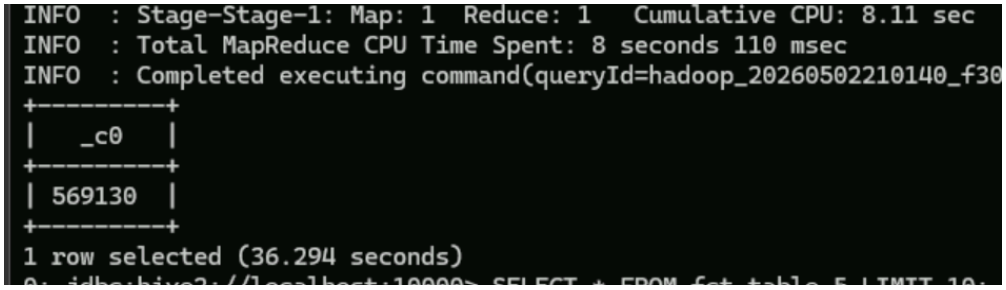
CREATE TABLE fct_table_5 (
    time_id INT,
    driver_id INT,
    severity_id INT,
    vehicle_id INT,
    severity_count INT
)
STORED AS PARQUET
TBLPROPERTIES (
    'parquet.compression'='SNAPPY',
    'skip.header.line.count'='1'
);

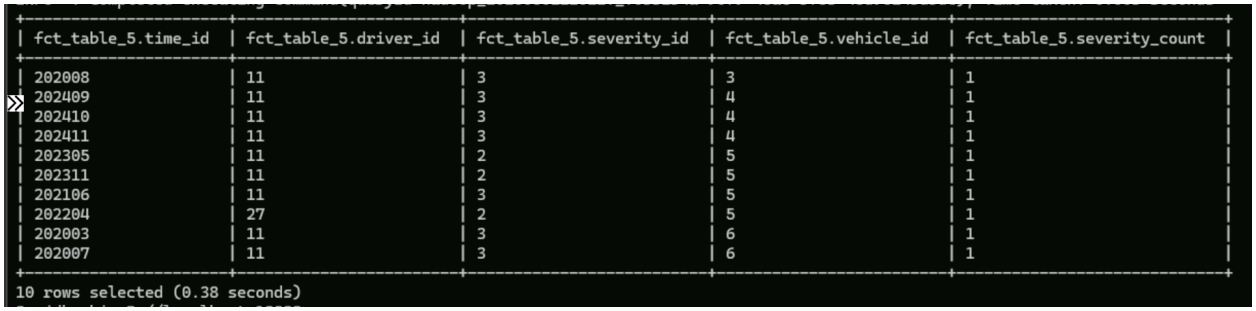
INSERT TABLE fct_table_5
SELECT
    t.time_id,
    dim_d.driver_id,
    dim_s.severity_id,
    dim_v.vehicle_id,
    COUNT(*) AS severity_count
FROM collisions co
JOIN vehicles v
    ON v.collision_index = co.collision_index
JOIN dim_time_temp t
    ON co.collision_date = t.collision_date
JOIN dim_driver dim_d
    ON v.age_band_of_driver = dim_d.age_band_of_driver
   AND v.sex_of_driver = dim_d.sex_of_driver
JOIN dim_severity dim_s

```
   ON co.collision_severity = dim_s.collision_severity
JOIN dim_vehicle dim_v
   ON v.vehicle_type = dim_v.vehicle_type
  AND v.age_of_vehicle = dim_v.age_of_vehicle
  AND (
      v.engine_capacity_bands = dim_v.engine_capacity_bands
      OR (v.engine_capacity_bands IS NULL AND dim_v.engine_capacity_bands IS NULL)
  )
  AND v.work_or_personal = dim_v.work_or_personal
  AND (
      v.generic_make_model = dim_v.generic_make_model
      OR (v.generic_make_model IS NULL AND dim_v.generic_make_model IS NULL)
  )
GROUP BY
   t.time_id,
   dim_d.driver_id,
   dim_s.severity_id,
   dim_v.vehicle_id;
```

Vet fact_5 table:

```
SELECT COUNT(*) FROM fct_table_5;
SELECT * FROM fct_table_5 LIMIT 10;
```





fct_table_5.time_id	fct_table_5.driver_id	fct_table_5.severity_id	fct_table_5.vehicle_id	fct_table_5.severity_count
202008	11	3	3	1
202409	11	3	4	1
202410	11	3	4	1
202411	11	3	4	1
202305	11	2	5	1
202311	11	2	5	1
202106	11	3	5	1
202204	27	2	5	1
202003	11	3	6	1
202007	11	3	6	1

10 rows selected (0.38 seconds)

.....

STORING META-DATA DOCUMENTATION IN HIVE:

Metadata documentation in Hive is used to describe table relationships, keys, and constraints. Although Hive does not strictly enforce primary and foreign keys, storing them as metadata helps document the database structure and shows how fact tables are linked to dimension tables.

ALTER TABLE fct_table_1 SET TBLPROPERTIES (

'constraints.primary.key'='PRIMARY KEY (time_id, area_id, condition_id, vehicle_id, severity_id)',

'constraints.fk_time1'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',

'constraints.fk_area1'='FOREIGN KEY (area_id) REFERENCES dim_area(area_id)',

'constraints.fk_condition1'='FOREIGN KEY (condition_id) REFERENCES dim_condition(condition_id)',

'constraints.fk_vehicle1'='FOREIGN KEY (vehicle_id) REFERENCES dim_vehicle(vehicle_id)',

'constraints.fk_severity1'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)'

);

ALTER TABLE fct_table_2 SET TBLPROPERTIES (

'constraints.primary.key'='PRIMARY KEY (time_id, driver_id, condition_id, severity_id)',

'constraints.fk_time2'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',

'constraints.fk_driver2'='FOREIGN KEY (driver_id) REFERENCES dim_driver(driver_id)',

'constraints.fk_condition2'='FOREIGN KEY (condition_id) REFERENCES dim_condition(condition_id)',

'constraints.fk_severity2'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)'

);

ALTER TABLE fct_table_3 SET TBLPROPERTIES (

'constraints.primary.key'='PRIMARY KEY (time_id, area_id, vehicle_id, severity_id)',

'constraints.fk_time3'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',

'constraints.fk_area3'='FOREIGN KEY (area_id) REFERENCES dim_area(area_id)',

'constraints.fk_vehicle3'='FOREIGN KEY (vehicle_id) REFERENCES dim_vehicle(vehicle_id)',

'constraints.fk_severity3'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)'

);

ALTER TABLE fct_table_4 SET TBLPROPERTIES (

'constraints.primary.key'='PRIMARY KEY (time_id, condition_id, vehicle_id, area_id, severity_id)',

'constraints.fk_time4'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',

'constraints.fk_condition4'='FOREIGN KEY (condition_id) REFERENCES dim_condition(condition_id)',

'constraints.fk_vehicle4'='FOREIGN KEY (vehicle_id) REFERENCES dim_vehicle(vehicle_id)',

'constraints.fk_area4'='FOREIGN KEY (area_id) REFERENCES dim_area(area_id)',

'constraints.fk_severity4'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)'

);

ALTER TABLE fct_table_5 SET TBLPROPERTIES (

'constraints.primary.key'='PRIMARY KEY (time_id, driver_id, vehicle_id, severity_id)',

'constraints.fk_time5'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',

'constraints.fk_driver5'='FOREIGN KEY (driver_id) REFERENCES dim_driver(driver_id)',

'constraints.fk_vehicle5'='FOREIGN KEY (vehicle_id) REFERENCES dim_vehicle(vehicle_id)',

'constraints.fk_severity5'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)'

);

After running the keys relationship documentation above:

Below, we fully demonstrate all schema on hive before building our schema on dbschema:

SHOW CREATE TABLE fct_table_1;

```
+----------------------------------------------------+
| CREATE EXTERNAL TABLE `fct_table_1`(               |
|   `time_id` int,                                   |
|   `area_id` int,                                   |
|   `condition_id` int,                              |
|   `vehicle_id` int,                                |
|   `severity_id` int,                               |
|   `severity_count` int)                            |
| ROW FORMAT SERDE                                   |
|   'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe'  |
| STORED AS INPUTFORMAT                              |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat'  |
| OUTPUTFORMAT                                       |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat' |
| LOCATION                                           |
|   'hdfs://localhost:9000/rta_olap/fct_table_1'     |
| TBLPROPERTIES (                                    |
|   'TRANSLATED_TO_EXTERNAL'='TRUE',                 |
|   'bucketing_version'='2',                         |
|   'constraints.fk_area1'='FOREIGN KEY (area_id) REFERENCES dim_area(area_id)',  |
|   'constraints.fk_condition1'='FOREIGN KEY (condition_id) REFERENCES dim_condition(condition_id)',  |
|   'constraints.fk_severity1'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)',  |
|   'constraints.fk_time1'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',  |
|   'constraints.fk_vehicle1'='FOREIGN KEY (vehicle_id) REFERENCES dim_vehicle(vehicle_id)',  |
|   'constraints.primary.key'='PRIMARY KEY (time_id, area_id, condition_id, vehicle_id, severity_id)',  |
|   'external.table.purge'='TRUE',                   |
|   'last_modified_by'='hiveuser',                   |
|   'last_modified_time'='1777759207',               |
|   'parquet.compression'='SNAPPY',                  |
|   'skip.header.line.count'='1',                    |
|   'transient_lastDdlTime'='1777759207')            |
+----------------------------------------------------+
```

SHOW CREATE TABLE fct_table_2;

```
INFO  : Starting task [Stage-0:DDL] in serial mode
INFO  : Completed executing command(queryId=hadoop_20260502220116_7c37a70e-0308-43c9-a505-58d144b47a6d); Time taken: 0.129 sec
+----------------------------------------------------+
|                   createtab_stmt                   |
+----------------------------------------------------+
| CREATE EXTERNAL TABLE `fct_table_2`(               |
|   `time_id` int,                                   |
|   `driver_id` int,                                 |
|   `condition_id` int,                              |
|   `severity_id` int,                               |
|   `severity_count` int)                            |
| ROW FORMAT SERDE                                   |
|   'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe'  |
| STORED AS INPUTFORMAT                              |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat'  |
| OUTPUTFORMAT                                       |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat' |
| LOCATION                                           |
|   'hdfs://localhost:9000/rta_olap/fct_table_2'     |
| TBLPROPERTIES (                                    |
|   'TRANSLATED_TO_EXTERNAL'='TRUE',                 |
|   'bucketing_version'='2',                         |
|   'constraints.fk_condition2'='FOREIGN KEY (condition_id) REFERENCES dim_condition(condition_id)',  |
|   'constraints.fk_driver2'='FOREIGN KEY (driver_id) REFERENCES dim_driver(driver_id)',  |
|   'constraints.fk_severity2'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)',  |
|   'constraints.fk_time2'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',  |
|   'constraints.primary.key'='PRIMARY KEY (time_id, driver_id, condition_id, severity_id)',  |
|   'external.table.purge'='TRUE',                   |
|   'last_modified_by'='hiveuser',                   |
|   'last_modified_time'='1777759208',               |
|   'parquet.compression'='SNAPPY',                  |
|   'skip.header.line.count'='1',                    |
|   'transient_lastDdlTime'='1777759208')            |
+----------------------------------------------------+
```

SHOW CREATE TABLE fct_table_3;

```
INFO  : Starting task [Stage-0:DDL] in serial mode
INFO  : Completed executing command(queryId=hadoop_20260502220116_cd24be9e-4b80-4d8d-b641-3c47413674d5); Time taken: 0.113 s
+----------------------------------------------------+
|                   createtab_stmt                   |
+----------------------------------------------------+
| CREATE EXTERNAL TABLE `fct_table_3`(               |
|   `time_id` int,                                   |
|   `area_id` int,                                   |
|   `vehicle_id` int,                                |
|   `severity_id` int,                               |
|   `severity_count` int)                            |
| ROW FORMAT SERDE                                   |
|   'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe'  |
| STORED AS INPUTFORMAT                              |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat'  |
| OUTPUTFORMAT                                       |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat' |
| LOCATION                                           |
|   'hdfs://localhost:9000/rta_olap/fct_table_3'     |
| TBLPROPERTIES (                                    |
|   'TRANSLATED_TO_EXTERNAL'='TRUE',                 |
|   'bucketing_version'='2',                         |
|   'constraints.fk_area3'='FOREIGN KEY (area_id) REFERENCES dim_area(area_id)',  |
|   'constraints.fk_severity3'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)',  |
|   'constraints.fk_time3'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',  |
|   'constraints.fk_vehicle3'='FOREIGN KEY (vehicle_id) REFERENCES dim_vehicle(vehicle_id)',  |
|   'constraints.primary.key'='PRIMARY KEY (time_id, area_id, vehicle_id, severity_id)',  |
|   'external.table.purge'='TRUE',                   |
|   'last_modified_by'='hiveuser',                   |
|   'last_modified_time'='1777759208',               |
|   'parquet.compression'='SNAPPY',                  |
|   'skip.header.line.count'='1',                    |
|   'transient_lastDdlTime'='1777759208')            |
+----------------------------------------------------+
```

SHOW CREATE TABLE fct_table_4;

```
+----------------------------------------------------+
|                   createtab_stmt                   |
+----------------------------------------------------+
| CREATE EXTERNAL TABLE `fct_table_4`(               |
|   `time_id` int,                                   |
|   `condition_id` int,                              |
|   `vehicle_id` int,                                |
|   `severity_id` int,                               |
|   `area_id` int,                                   |
|   `severity_count` int)                            |
| ROW FORMAT SERDE                                   |
|   'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe'  |
| STORED AS INPUTFORMAT                              |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat'  |
| OUTPUTFORMAT                                       |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat' |
| LOCATION                                           |
|   'hdfs://localhost:9000/rta_olap/fct_table_4'     |
| TBLPROPERTIES (                                    |
|   'TRANSLATED_TO_EXTERNAL'='TRUE',                 |
|   'bucketing_version'='2',                         |
|   'constraints.fk_area4'='FOREIGN KEY (area_id) REFERENCES dim_area(area_id)',  |
|   'constraints.fk_condition4'='FOREIGN KEY (condition_id) REFERENCES dim_condition(condition_id)',  |
|   'constraints.fk_severity4'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)',  |
|   'constraints.fk_time4'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',  |
|   'constraints.fk_vehicle4'='FOREIGN KEY (vehicle_id) REFERENCES dim_vehicle(vehicle_id)',  |
|   'constraints.primary.key'='PRIMARY KEY (time_id, condition_id, vehicle_id, area_id, severity_id)',  |
|   'external.table.purge'='TRUE',                   |
|   'last_modified_by'='hiveuser',                   |
|   'last_modified_time'='1777759209',               |
|   'parquet.compression'='SNAPPY',                  |
|   'skip.header.line.count'='1',                    |
|   'transient_lastDdlTime'='1777759209')            |
+----------------------------------------------------+
30 rows selected (0.223 seconds)
```

SHOW CREATE TABLE fct_table_5;

```
|                  createtab_stmt                  |
+--------------------------------------------------+
| CREATE EXTERNAL TABLE `fct_table_5`(             |
|   `time_id` int,                                 |
|   `driver_id` int,                               |
|   `severity_id` int,                             |
|   `vehicle_id` int,                              |
|   `severity_count` int)                          |
| ROW FORMAT SERDE                                 |
|   'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe'  |
| STORED AS INPUTFORMAT                            |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat'  |
| OUTPUTFORMAT                                     |
|   'org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat' |
| LOCATION                                         |
|   'hdfs://localhost:9000/rta_olap/fct_table_5'   |
| TBLPROPERTIES (                                  |
|   'TRANSLATED_TO_EXTERNAL'='TRUE',               |
|   'bucketing_version'='2',                       |
|   'constraints.fk_driver5'='FOREIGN KEY (driver_id) REFERENCES dim_driver(driver_id)',  |
|   'constraints.fk_severity5'='FOREIGN KEY (severity_id) REFERENCES dim_severity(severity_id)',  |
|   'constraints.fk_time5'='FOREIGN KEY (time_id) REFERENCES dim_time(time_id)',  |
|   'constraints.fk_vehicle5'='FOREIGN KEY (vehicle_id) REFERENCES dim_vehicle(vehicle_id)',  |
|   'constraints.primary.key'='PRIMARY KEY (time_id, driver_id, vehicle_id, severity_id)',  |
|   'external.table.purge'='TRUE',                 |
|   'last_modified_by'='hiveuser',                 |
|   'last_modified_time'='1777759249',             |
|   'parquet.compression'='SNAPPY',                |
|   'skip.header.line.count'='1',                  |
|   'transient_lastDdlTime'='1777759249')          |
+--------------------------------------------------+
28 rows selected (0.243 seconds)
```

……..

SAMPLE OF SCHEMA IN DBSCHEMA:

fct_table_1:

Primary Key: time_id, area_id, condition_id, vehicle_id, severity_id

Foreign Keys:

time_id ──└── dim_time(time_id)

area_id ──└── dim_area(area_id)

Condition_id ──└── dim_condition(condition_id)

vehicle_id ──└── dim_vehicle(vehicle_id)

severity_id ──└── dim_severity(severity_id)

……..

fct_table_2

Primary Key: time_id, driver_id, condition_id, severity_id

Foreign Keys:

time_id ──└── dim_time(time_id)

driver_id ──└── dim_driver(driver_id)

condition_id ──└── dim_condition(condition_id)

severity_id ──└── dim_severity(severity_id)

……..

fct_table_3

Primary Key: time_id, area_id, vehicle_id, severity_id

Foreign Keys: time_id ──└── dim_time(time_id)

area_id ──└── dim_area(area_id)

vehicle_id ──└── dim_vehicle(vehicle_id)

severity_id ──└── dim_severity(severity_id)

……..

fct_table_4

Primary Key: time_id, condition_id, vehicle_id, area_id, severity_id

Foreign Keys:

time_id ──└── dim_time(time_id)

condition_id ──└── dim_condition(condition_id)

vehicle_id ──└── dim_vehicle(vehicle_id)

area_id ──└── dim_area(area_id)

severity_id ──└── dim_severity(severity_id)

……..

fct_table_5

Primary Key: time_id, driver_id, vehicle_id, severity_id

Foreign Keys:

time_id ──└── dim_time(time_id)

driver_id ──└── dim_driver(driver_id)

vehicle_id ──└── dim_vehicle(vehicle_id)

severity_id ──└── dim_severity(severity_id)

……..

All fact tables connect to:

dim_time (when)

dim_severity (how severe)

dim_vehicle (what vehicle)

dim_driver (who)

dim_area (where)

dim_condition (conditions)

……

……..

After the above, we can export to DBSchema for the graphical relationship schema design.

……

ANSWERING BUSINESS QUESTIONS

Business Question 1:

Provide monthly and yearly breakdown of the number of casualty severities by vehicle type, vehicle age, and urban vs. Rural areas in the UK from 2020 to 2024.

1a. Monthly Breakdown of highest number of casualty severities between urban and rural:

This query calculates the total number of collisions per month, grouped by year, urban and rural areas, and collision severity. The results provide a monthly breakdown to analyse how collision severity patterns vary between urban and rural areas over time.

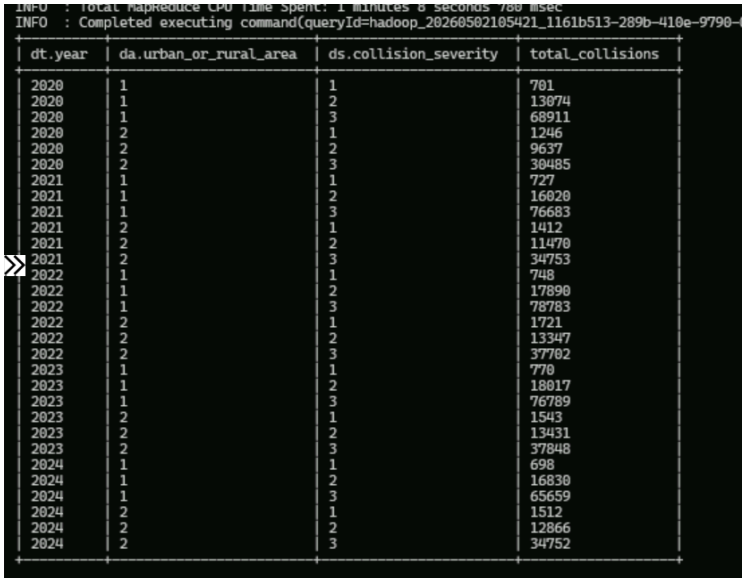
```
SELECT
    dt.year,
    dt.month,
    da.urban_or_rural_area,
    dv.vehicle_type,
    dv.age_of_vehicle,
    ds.collision_severity,
    SUM(f1.severity_count) AS total_collisions
FROM fct_table_1 f1
JOIN dim_time dt
    ON f1.time_id = dt.time_id
JOIN dim_area da
    ON f1.area_id = da.area_id
JOIN dim_vehicle dv
    ON f1.vehicle_id = dv.vehicle_id
JOIN dim_severity ds
    ON f1.severity_id = ds.severity_id
WHERE dv.age_of_vehicle BETWEEN 0 AND 30
GROUP BY
    dt.year,
    dt.month,
    da.urban_or_rural_area,
    dv.vehicle_type,
    dv.age_of_vehicle,
    ds.collision_severity
ORDER BY
    dt.year,
    dt.month,
    da.urban_or_rural_area,
    dv.vehicle_type,
    dv.age_of_vehicle,
    ds.collision_severity;
```

dt.year	dt.month	da.urban_or_rural_area	dv.vehicle_type	dv.age_of_vehicle	ds.collision_severity	total_collisions
2024	12	2	8	20	2	1
2024	12	2	8	20	3	1
2024	12	2	8	21	2	1
2024	12	2	8	21	3	3
2024	12	2	8	24	3	1
2024	12	2	8	29	2	1
2024	12	2	8	29	3	3
2024	12	2	9	0	1	9
2024	12	2	9	0	2	78
2024	12	2	9	0	3	217
2024	12	2	9	1	1	5
2024	12	2	9	1	2	79
2024	12	2	9	1	3	289
2024	12	2	9	2	1	9
2024	12	2	9	2	2	103
2024	12	2	9	2	3	261
2024	12	2	9	3	1	10
2024	12	2	9	3	2	84
2024	12	2	9	3	3	239
2024	12	2	9	4	1	7
2024	12	2	9	4	2	68

1b. Yearly breakdown of highest number of severities between urban and rural:

This query calculates the total number of collisions per year, grouped by urban and rural areas and collision severity. The results provide a yearly breakdown to analyse how collision severity levels vary between urban and rural areas on yearly basis.

```
SELECT
    dt.year,
    da.urban_or_rural_area,
    ds.collision_severity,
    SUM(f1.severity_count) AS total_collisions
FROM fct_table_1 f1
JOIN dim_time dt
    ON f1.time_id = dt.time_id
JOIN dim_area da
    ON f1.area_id = da.area_id
JOIN dim_severity ds
    ON f1.severity_id = ds.severity_id
GROUP BY
    dt.year,
    da.urban_or_rural_area,
    ds.collision_severity
ORDER BY
    dt.year,
    da.urban_or_rural_area,
    ds.collision_severity;
```

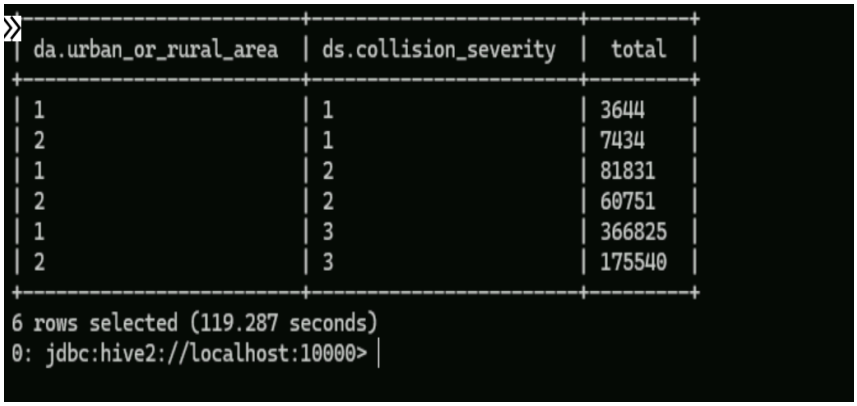


```
INFO  : Total MapReduce CPU Time Spent: 1 minutes 8 seconds 780 msec
INFO  : Completed executing command(queryId=hadoop_20260502105421_1161b513-289b-410e-9790-
+-----------+-------------------------+--------------------------+--------------------+
| dt.year   | da.urban_or_rural_area  | ds.collision_severity    | total_collisions   |
+-----------+-------------------------+--------------------------+--------------------+
| 2020      | 1                       | 1                        | 701                |
| 2020      | 1                       | 2                        | 13074              |
| 2020      | 1                       | 3                        | 68911              |
| 2020      | 2                       | 1                        | 1246               |
| 2020      | 2                       | 2                        | 9637               |
| 2020      | 2                       | 3                        | 30485              |
| 2021      | 1                       | 1                        | 727                |
| 2021      | 1                       | 2                        | 16020              |
| 2021      | 1                       | 3                        | 76683              |
| 2021      | 2                       | 1                        | 1412               |
| 2021      | 2                       | 2                        | 11470              |
| 2021      | 2                       | 3                        | 34753              |
| 2022      | 1                       | 1                        | 748                |
| 2022      | 1                       | 2                        | 17890              |
| 2022      | 1                       | 3                        | 78783              |
| 2022      | 2                       | 1                        | 1721               |
| 2022      | 2                       | 2                        | 13347              |
| 2022      | 2                       | 3                        | 37702              |
| 2023      | 1                       | 1                        | 770                |
| 2023      | 1                       | 2                        | 18017              |
| 2023      | 1                       | 3                        | 76789              |
| 2023      | 2                       | 1                        | 1543               |
| 2023      | 2                       | 2                        | 13431              |
| 2023      | 2                       | 3                        | 37848              |
| 2024      | 1                       | 1                        | 698                |
| 2024      | 1                       | 2                        | 16830              |
| 2024      | 1                       | 3                        | 65659              |
| 2024      | 2                       | 1                        | 1512               |
| 2024      | 2                       | 2                        | 12866              |
| 2024      | 2                       | 3                        | 34752              |
+-----------+-------------------------+--------------------------+--------------------+
```

1c. Highest severity overall between urban and rural:

This query calculates the total number of collisions by time period, urban or rural area, and collision severity. The results are grouped by these attributes to compare how collision severity varies between urban and rural areas between the year 2021 -2024.

```
SELECT dt.time_id, dv.vehicle_type, dv.age_of_vehicle, da.urban_or_rural_area,
ds.collision_severity
SELECT
    dt.time_id,
    da.urban_or_rural_area,
    ds.collision_severity,
    SUM(f1.severity_count) AS no_of_collisions
FROM fct_table_1 f1
JOIN dim_time dt ON f1.time_id = dt.time_id
JOIN dim_area da ON f1.area_id = da.area_id
JOIN dim_severity ds ON f1.severity_id = ds.severity_id
GROUP BY
    dt.time_id,
    da.urban_or_rural_area,
    ds.collision_severity
ORDER BY dt.time_id;
```



da.urban_or_rural_area	ds.collision_severity	total
1	1	3644
2	1	7434
1	2	81831
2	2	60751
1	3	366825
2	3	175540

6 rows selected (119.287 seconds)
0: jdbc:hive2://localhost:10000>

……

Business Question 2:

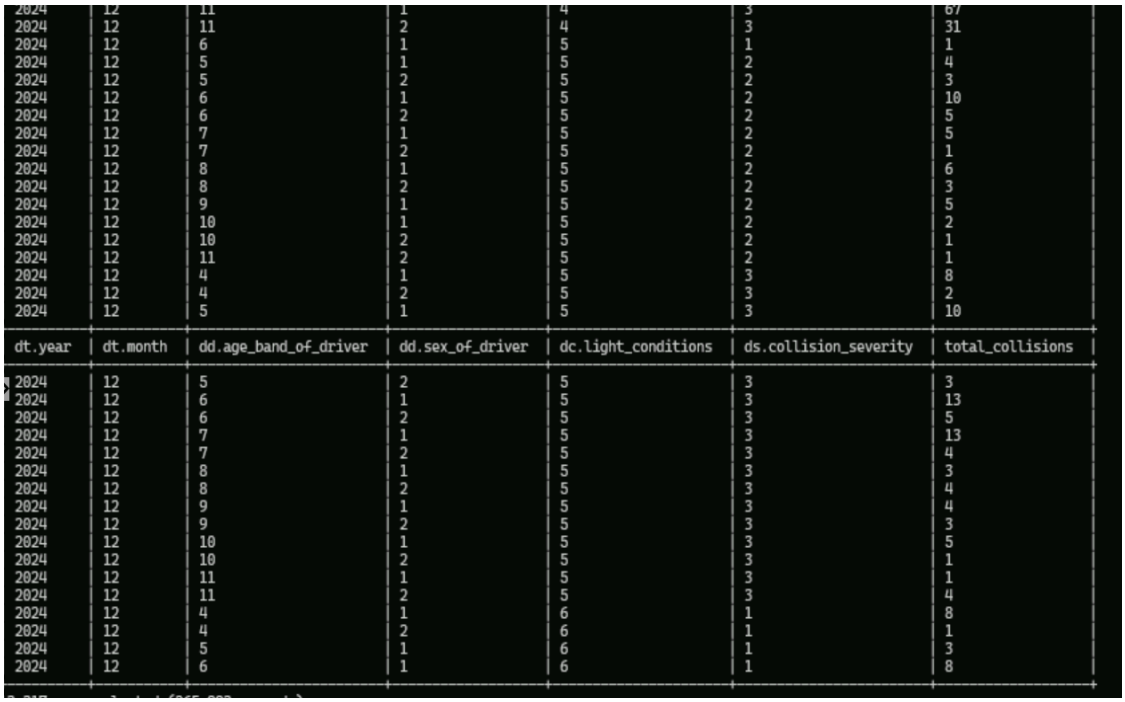
Provide a monthly and yearly breakdown of the number of collision severities by driver age group, sex of driver and light conditions in the UK between 2020 - 2024.

2a Monthly Breakdown:

This query calculates the total number of collisions by year, month, driver age band, driver gender, light conditions, and collision severity. The results are grouped by these attributes to analyse how collision patterns vary across different driver demographics, lighting conditions, severity levels, and over time at a monthly basis.

```
SELECT
    dt.year,
    dt.month,
    dd.age_band_of_driver,
    dd.sex_of_driver,
    dc.light_conditions,
    ds.collision_severity,
    SUM(f2.severity_count) AS total_collisions
FROM fct_table_2 f2
JOIN dim_time dt
    ON f2.time_id = dt.time_id
JOIN dim_driver dd
    ON f2.driver_id = dd.driver_id
JOIN dim_condition dc
    ON f2.condition_id = dc.condition_id
JOIN dim_severity ds
    ON f2.severity_id = ds.severity_id
GROUP BY
    dt.year,
    dt.month,
    dd.age_band_of_driver,
```

```
    dd.sex_of_driver,
    dc.light_conditions,
    ds.collision_severity
ORDER BY
    dt.year,
    dt.month;
```

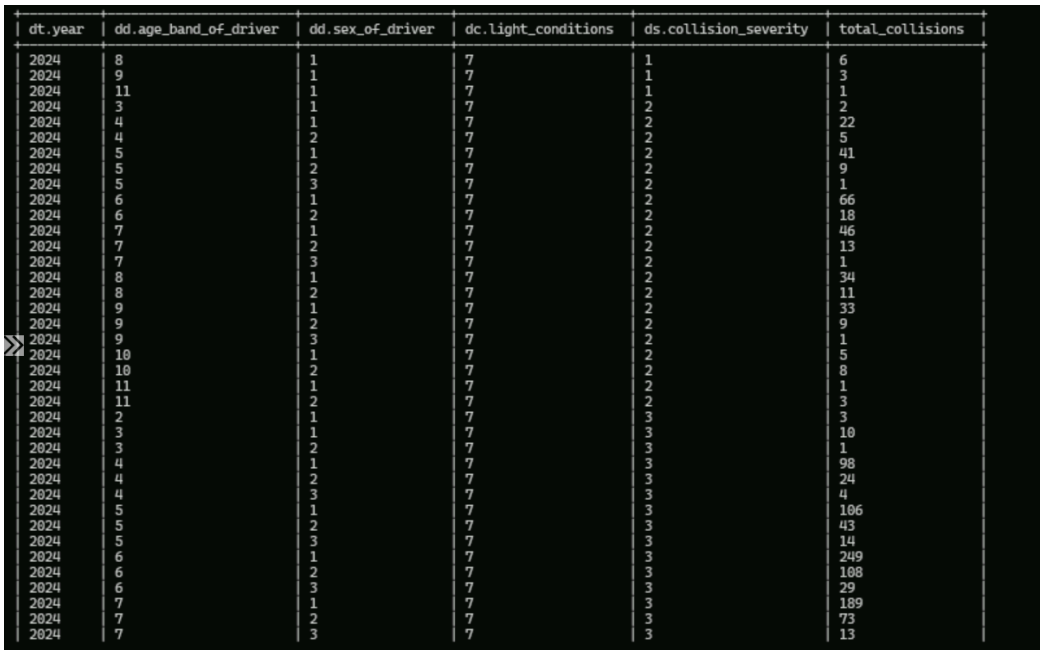


dt.year	dt.month	dd.age_band_of_driver	dd.sex_of_driver	dc.light_conditions	ds.collision_severity	total_collisions
2024	12	11	1	4	3	67
2024	12	11	2	4	3	31
2024	12	6	1	5	1	1
2024	12	5	1	5	2	4
2024	12	5	2	5	2	3
2024	12	6	1	5	2	10
2024	12	6	2	5	2	5
2024	12	7	1	5	2	5
2024	12	7	2	5	2	1
2024	12	8	1	5	2	6
2024	12	8	2	5	2	3
2024	12	9	1	5	2	5
2024	12	10	1	5	2	2
2024	12	10	2	5	2	1
2024	12	11	2	5	2	1
2024	12	4	1	5	3	8
2024	12	4	2	5	3	2
2024	12	5	1	5	3	10
2024	12	5	2	5	3	3
2024	12	6	1	5	3	13
2024	12	6	2	5	3	5
2024	12	7	1	5	3	13
2024	12	7	2	5	3	4
2024	12	8	1	5	3	3
2024	12	8	2	5	3	4
2024	12	9	1	5	3	4
2024	12	9	2	5	3	3
2024	12	10	1	5	3	5
2024	12	10	2	5	3	1
2024	12	11	1	5	3	1
2024	12	11	2	5	3	4
2024	12	4	1	6	1	8
2024	12	4	2	6	1	1
2024	12	5	1	6	1	3
2024	12	6	1	6	1	8

2b.

```
SELECT
    dt.year,
    dd.age_band_of_driver,
    dd.sex_of_driver,
    dc.light_conditions,
    ds.collision_severity,
    SUM(f2.severity_count) AS total_collisions
FROM fct_table_2 f2
JOIN dim_time dt
    ON f2.time_id = dt.time_id
JOIN dim_driver dd
    ON f2.driver_id = dd.driver_id
JOIN dim_condition dc
    ON f2.condition_id = dc.condition_id
JOIN dim_severity ds
    ON f2.severity_id = ds.severity_id
```

```
GROUP BY
    dt.year,
    dd.age_band_of_driver,
    dd.sex_of_driver,
    dc.light_conditions,
    ds.collision_severity
ORDER BY
    dt.year;
```

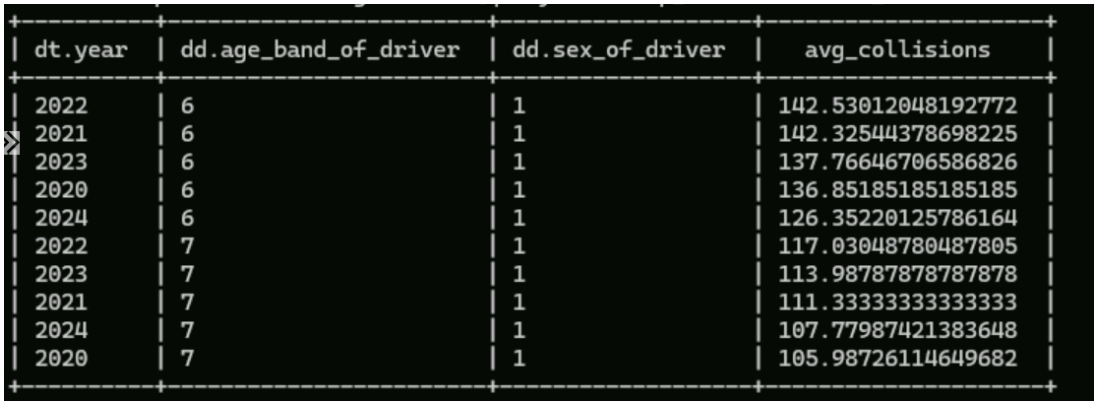


dt.year	dd.age_band_of_driver	dd.sex_of_driver	dc.light_conditions	ds.collision_severity	total_collisions
2024	8	1	7	1	6
2024	9	1	7	1	3
2024	11	1	7	1	1
2024	3	1	7	2	2
2024	4	1	7	2	22
2024	4	2	7	2	5
2024	5	1	7	2	41
2024	5	2	7	2	9
2024	5	3	7	2	1
2024	6	1	7	2	66
2024	6	2	7	2	18
2024	7	1	7	2	46
2024	7	2	7	2	13
2024	7	3	7	2	1
2024	8	1	7	2	34
2024	8	2	7	2	11
2024	9	1	7	2	33
2024	9	2	7	2	9
2024	9	3	7	2	1
2024	10	1	7	2	5
2024	10	2	7	2	8
2024	11	1	7	2	1
2024	11	2	7	2	3
2024	2	1	7	3	3
2024	3	1	7	3	10
2024	3	2	7	3	1
2024	4	1	7	3	98
2024	4	2	7	3	24
2024	4	3	7	3	4
2024	5	1	7	3	106
2024	5	2	7	3	43
2024	5	3	7	3	14
2024	6	1	7	3	249
2024	6	2	7	3	108
2024	6	3	7	3	29
2024	7	1	7	3	189
2024	7	2	7	3	73
2024	7	3	7	3	13

2c. This query calculates the average number of collisions per year by driver age band and gender across various location. The results are grouped by these attributes to analyse how collision patterns vary across different age groups and genders over time.

```
SELECT
    dt.year,
    dd.age_band_of_driver,
    dd.sex_of_driver,
    AVG(f2.severity_count) AS avg_collisions
FROM fct_table_2 f2
JOIN dim_time dt
    ON f2.time_id = dt.time_id
JOIN dim_driver dd
    ON f2.driver_id = dd.driver_id
GROUP BY
    dt.year,
```

```
    dd.age_band_of_driver,
    dd.sex_of_driver
ORDER BY
    avg_collisions DESC
LIMIT 10;
```



dt.year	dd.age_band_of_driver	dd.sex_of_driver	avg_collisions
2022	6	1	142.53012048192772
2021	6	1	142.32544378698225
2023	6	1	137.76646706586826
2020	6	1	136.85185185185185
2024	6	1	126.35220125786164
2022	7	1	117.03048780487805
2023	7	1	113.98787878787878
2021	7	1	111.33333333333333
2024	7	1	107.77987421383648
2020	7	1	105.98726114649682

Business Question 3:

Who were the top 30 worst vehicle make & models by fatal collisions across urban and rural areas for each month and year in the UK between 2020 - 2024.

3a. Yearly and Monthly Break-down: Models of Cars

This query calculates the total number of fatal collisions by vehicle make and model over time. The results are grouped by time period and vehicle type, and it returns the top 30 vehicle models with the highest number of fatal collisions.

```
SELECT
  dt.year,
  dt.month,
  da.urban_or_rural_area,
  dv.generic_make_model,
  SUM(f3.severity_count) AS total_fatal_collisions
FROM fct_table_3 f3
JOIN dim_time dt ON f3.time_id = dt.time_id
JOIN dim_vehicle dv ON f3.vehicle_id = dv.vehicle_id
JOIN dim_area da ON f3.area_id = da.area_id
JOIN dim_severity ds ON f3.severity_id = ds.severity_id
WHERE ds.collision_severity = 1
  AND dv.generic_make_model IS NOT NULL
  AND dt.year BETWEEN 2020 AND 2024
GROUP BY
  dt.year,
```

```
  dt.month,
  da.urban_or_rural_area,
  dv.generic_make_model
ORDER BY total_fatal_collisions DESC
LIMIT 30;
```

dt.year	dt.month	dv.generic_make_model	ds.collision_severity	sum_of_fatal_collisions
2022	11	YAMAHA MODEL MISSING	1	8
2024	11	CITROEN C4	1	7
2024	10	VAUXHALL ZAFIRA	1	7
2022	1	VAUXHALL ASTRA	1	7
2023	5	YAMAHA MODEL MISSING	1	6
2022	1	YAMAHA MODEL MISSING	1	6
2021	7	YAMAHA MODEL MISSING	1	6
2020	1	PEUGEOT 3008	1	6
2022	1	TOYOTA AVENSIS	1	6
2024	8	VAUXHALL INSIGNIA	1	5
2021	5	YAMAHA YZF	1	5
2022	9	HONDA MODEL MISSING	1	5
2020	2	HONDA MODEL MISSING	1	5
2021	8	TOYOTA MODEL MISSING	1	5
2022	9	SUZUKI Model Missing	1	5
2022	12	HARLEY-DAVIDSON MODEL MISSING	1	5
2023	10	AUDI A3	1	5
2023	8	PEUGEOT 2008	1	5
2023	7	KTM EXC	1	5
2024	11	HYUNDAI I30	1	5
2023	8	YAMAHA YZF	1	5

3b. Yearly Breakdown:
This result presents vehicle models with the number of fatal collisions, segmented by year and area type (urban or rural). This gives an insight on vehicles associated with the highest number of fatal collisions over time.

```
SELECT
dt.year,
da.urban_or_rural_area,
dv.generic_make_model,
SUM(f3.severity_count) AS total_fatal_collisions
FROM fct_table_3 f3
JOIN dim_time dt ON f3.time_id = dt.time_id
JOIN dim_vehicle dv ON f3.vehicle_id = dv.vehicle_id
JOIN dim_area da ON f3.area_id = da.area_id
JOIN dim_severity ds ON f3.severity_id = ds.severity_id
WHERE ds.collision_severity = 1
AND dv.generic_make_model IS NOT NULL
AND dt.year BETWEEN 2020 AND 2024
GROUP BY
dt.year,
da.urban_or_rural_area,
dv.generic_make_model
```

ORDER BY total_fatal_collisions DESC
LIMIT 30;

dt.year	da.urban_or_rural_area	dv.generic_make_model	total_fatal_collisions
2022	2	YAMAHA MODEL MISSING	22
2024	2	SUZUKI MODEL MISSING	20
2022	2	TOYOTA AVENSIS	19
2023	2	AUDI A3	19
2021	2	HONDA MODEL MISSING	18
2022	2	VAUXHALL ASTRA	18
2022	2	SUZUKI GSXR	18
2024	2	HONDA MODEL MISSING	17
2023	2	YAMAHA YZF	17
2022	2	VOLKSWAGEN PASSAT	17
2024	2	CITROEN C4	17
2022	2	HARLEY-DAVIDSON MODEL MISSING	16
2023	2	SUZUKI GSXR	16
2021	2	AUDI A3	16
2023	2	YAMAHA MODEL MISSING	16
2022	1	YAMAHA MODEL MISSING	15
2021	2	YAMAHA MODEL MISSING	15
2022	2	AUDI A3	15
2022	2	VAUXHALL ZAFIRA	15
2022	2	SUZUKI MODEL MISSING	14
2022	2	YAMAHA Model Missing	14

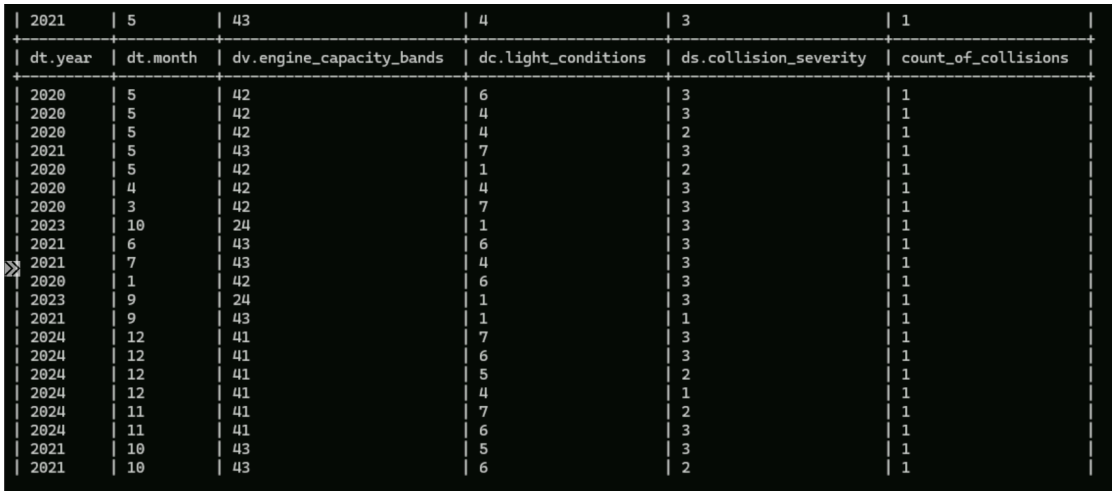
Business Question 4:

Provide a count of the number of collision severities by engine capacity, light conditions and month for cars between 2020 - 2024 in urban areas of the UK?

4a. This query demonstrate the count of collisions by engine capacity, light conditions, and month for cars between 2020 and 2024 in urban areas of the UK. The results are grouped by these attributes to analyse how collision severity varies across different vehicle engine sizes, lighting conditions, and over time.

```
SELECT
  dt.year,
  dt.month,
  dv.engine_capacity_bands,
  dc.light_conditions,
  ds.collision_severity,
  SUM(f4.severity_count) AS count_of_collisions
FROM fct_table_4 f4
JOIN dim_vehicle dv ON f4.vehicle_id = dv.vehicle_id
JOIN dim_time dt ON f4.time_id = dt.time_id
JOIN dim_severity ds ON f4.severity_id = ds.severity_id
JOIN dim_area da ON f4.area_id = da.area_id
JOIN dim_condition dc ON f4.condition_id = dc.condition_id
```

```
WHERE (dv.vehicle_type = 8 OR dv.vehicle_type = 9)
  AND da.urban_or_rural_area = 1
  AND dt.year BETWEEN 2020 AND 2024
GROUP BY
  dt.year,
  dt.month,
  dv.engine_capacity_bands,
  dc.light_conditions,
  ds.collision_severity
ORDER BY count_of_collisions DESC
LIMIT 10;
```



dt.year	dt.month	dv.engine_capacity_bands	dc.light_conditions	ds.collision_severity	count_of_collisions
2020	5	42	6	3	1
2020	5	42	4	3	1
2020	5	42	4	2	1
2021	5	43	7	3	1
2020	5	42	1	2	1
2020	4	42	4	3	1
2020	3	42	7	3	1
2023	10	24	1	3	1
2021	6	43	6	3	1
2021	7	43	4	3	1
2020	1	42	6	3	1
2023	9	24	1	3	1
2021	9	43	1	1	1
2024	12	41	7	3	1
2024	12	41	6	3	1
2024	12	41	5	2	1
2024	12	41	4	1	1
2024	11	41	7	2	1
2024	11	41	6	3	1
2021	10	43	5	3	1
2021	10	43	6	2	1

4b. Our second query calculates the average collision severity count by engine capacity, light conditions, and location. The results are grouped by these attributes to show how collision severity varies across different vehicle engine sizes, lighting conditions, and location.

```
SELECT
  da.urban_or_rural_area,
  dv.engine_capacity_bands,
  dc.light_conditions,
  ds.collision_severity,
  AVG(f4.severity_count) AS avg_collision_severity
FROM fct_table_4 f4
JOIN dim_vehicle dv ON f4.vehicle_id = dv.vehicle_id
JOIN dim_time dt ON f4.time_id = dt.time_id
JOIN dim_severity ds ON f4.severity_id = ds.severity_id
JOIN dim_area da ON f4.area_id = da.area_id
JOIN dim_condition dc ON f4.condition_id = dc.condition_id
WHERE (dv.vehicle_type = 8 OR dv.vehicle_type = 9)
```

```
  AND dt.year BETWEEN 2020 AND 2024
GROUP BY
  da.urban_or_rural_area,
  dv.engine_capacity_bands,
  dc.light_conditions,
  ds.collision_severity
ORDER BY avg_collision_severity DESC
LIMIT 30;
```

da.urban_or_rural_area	dv.engine_capacity_bands	dc.light_conditions	ds.collision_severity	avg_collision_severity
1	32	1	3	1.8839244587747581
1	33	1	3	1.6594085147871303
1	30	1	3	1.6370035193564605
1	37	1	3	1.5782691092861656
1	31	1	3	1.5554910779051212
1	35	1	3	1.5390132086605237
1	22	4	2	1.5
1	32	4	3	1.489974293059126
1	34	1	3	1.4721055788842232
1	41	1	3	1.44282744282744428
1	39	1	3	1.4326365777960588
1	37	4	3	1.3801147674132415
1	30	4	3	1.339734705998812
1	33	4	3	1.330628803245436
1	NULL	1	3	1.3124779073877695
1	40	1	3	1.3098591549295775
1	36	1	3	1.2908827785817656
1	40	4	3	1.2897196261682242
1	35	4	3	1.286085150571132
1	31	4	3	1.279045279045279
1	41	4	3	1.272408612919379

4c. This query calculates the total number of collisions by year, month, engine capacity, light conditions, and collision severity for cars in urban areas between 2020 and 2024. The results are grouped by these attributes to analyse how collision severity varies across different vehicle engine sizes, lighting conditions, and over time at a monthly level.

```
dt.year,
dt.month,
dv.engine_capacity_bands,
dc.light_conditions,
ds.collision_severity,
SUM(f4.severity_count) AS count_of_collisions
FROM fct_table_4 f4
JOIN dim_vehicle dv ON f4.vehicle_id = dv.vehicle_id
JOIN dim_time dt ON f4.time_id = dt.time_id
JOIN dim_severity ds ON f4.severity_id = ds.severity_id
JOIN dim_area da ON f4.area_id = da.area_id
JOIN dim_condition dc ON f4.condition_id = dc.condition_id
WHERE (dv.vehicle_type = 8 OR dv.vehicle_type = 9)
AND da.urban_or_rural_area = 1
AND dt.year BETWEEN 2020 AND 2024
GROUP BY
```

```
dt.year,
dt.month,
dv.engine_capacity_bands,
dc.light_conditions,
ds.collision_severity
ORDER BY count_of_collisions DESC
LIMIT 10;
```

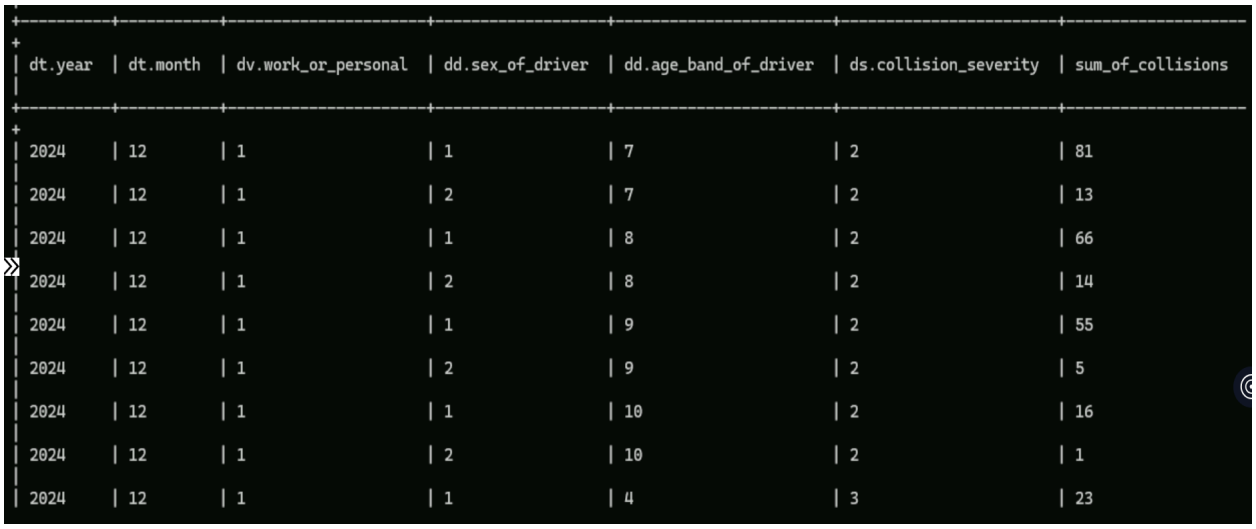
dt.year	dt.month	dv.engine_capacity_bands	dc.light_conditions	ds.collision_severity	count_of_collisions
2021	5	39	1	3	976
2021	7	39	1	3	939
2021	6	39	1	3	938
2022	5	39	1	3	927
2022	6	39	1	3	905
2023	6	39	1	3	902
2021	4	39	1	3	888
2021	5	35	1	3	866
2023	5	39	1	3	843
2021	9	39	1	3	829

QUESTION 5:
Provide the counts of collision_severity by work & personal use vehicles, sex & age group of driver and month from 2020 - 2024 in the UK?

This query calculates the total number of collisions per month between 2020 and 2024, grouped by vehicle usage (work or personal), driver gender, driver age group, and collision severity. It shows how collision severity varies across different driver demographics and vehicle usage types.

```
SELECT
  dt.year,
  dt.month,
  dv.work_or_personal,
  dd.sex_of_driver,
  dd.age_band_of_driver,
  ds.collision_severity,
  SUM(f2.severity_count) AS total_collisions
FROM fct_table_2 f2
JOIN dim_time dt
  ON f2.time_id = dt.time_id
JOIN dim_driver dd
  ON f2.driver_id = dd.driver_id
JOIN dim_vehicle dv
  ON f2.vehicle_id = dv.vehicle_id
JOIN dim_severity ds
  ON f2.severity_id = ds.severity_id
```

```
WHERE dt.year BETWEEN 2020 AND 2024
GROUP BY
  dt.year,
  dt.month,
  dv.work_or_personal,
  dd.sex_of_driver,
  dd.age_band_of_driver,
  ds.collision_severity
ORDER BY
  dt.year,
  dt.month;
```



dt.year	dt.month	dv.work_or_personal	dd.sex_of_driver	dd.age_band_of_driver	ds.collision_severity	sum_of_collisions
2024	12	1	1	7	2	81
2024	12	1	2	7	2	13
2024	12	1	1	8	2	66
2024	12	1	2	8	2	14
2024	12	1	1	9	2	55
2024	12	1	2	9	2	5
2024	12	1	1	10	2	16
2024	12	1	2	10	2	1
2024	12	1	1	4	3	23

……..